

智能计算方法

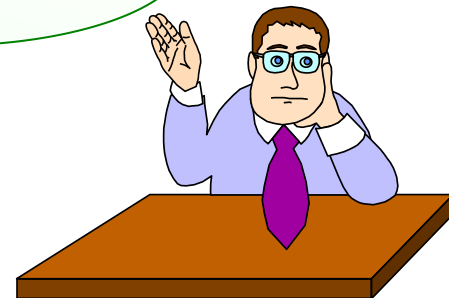
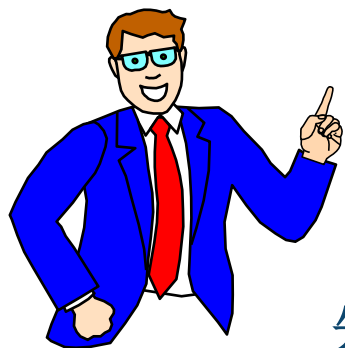
课程讲义

第六部分：其他智能计算方法

第一节：模拟退火

模拟退火算法思想

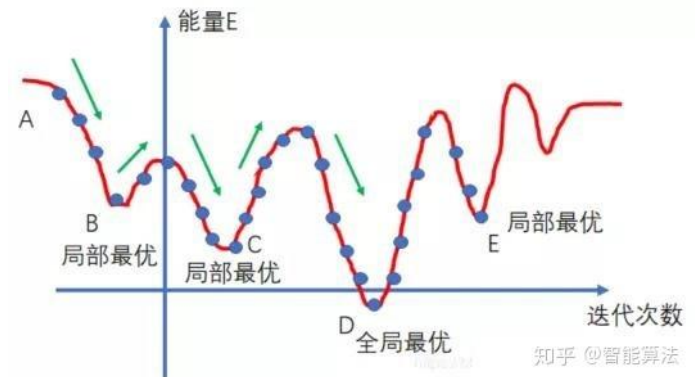
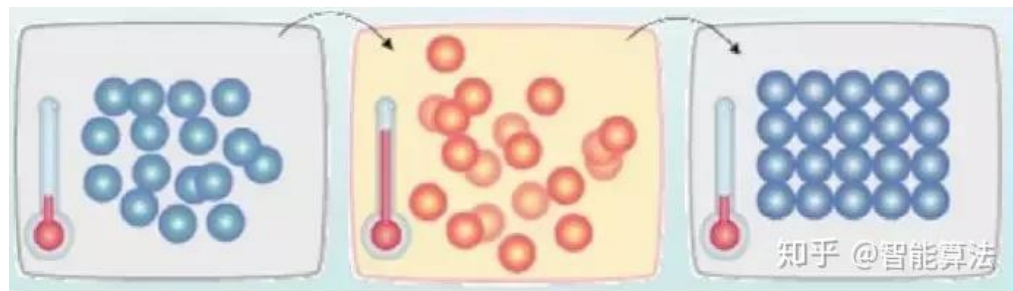
模拟退火算法（Simulated Annealing, SA）是一种模拟物理退火的过程而设计的优化算法。它的基本思想最早在1953年就被Metropolis提出，但直到1983年Kirkpatrick等才设计出真正意义上的模拟退火算法并进行应用。



模拟退火算法采用类似于物理退火的过程，先在一个高温状态下（相当于算法随机搜索），然后逐渐退火，在每个温度下（相当于算法的每一次状态转移）徐徐冷却（相当于算法局部搜索），最终达到物理基态（相当于算法找到最优解）。

模拟退火算法思想

- ❧ 模拟退火算法来源于固体退火原理，是一种基于概率的算法。
- ❧ 将固体加温至充分高，再让其徐徐冷却，加温时，固体内部粒子随温升变为无序状，内能增大，而徐徐冷却时粒子渐趋有序，在每个温度都达到平衡态，最后在常温时达到基态，内能减为最小。
- ❧ 模拟退火算法是通过赋予搜索过程一种时变且最终趋于零的概率突跳性，从而可有效避免陷入局部极小并最终趋于全局最优的串行结构的优化算法。



模拟退火算法思想

物理退火过程

物理退火过程

- 物体内部的状态
- 状态的能量
- 温度
- 熔解过程
- 退火冷却过程
- 状态的转移
- 能量最低状态

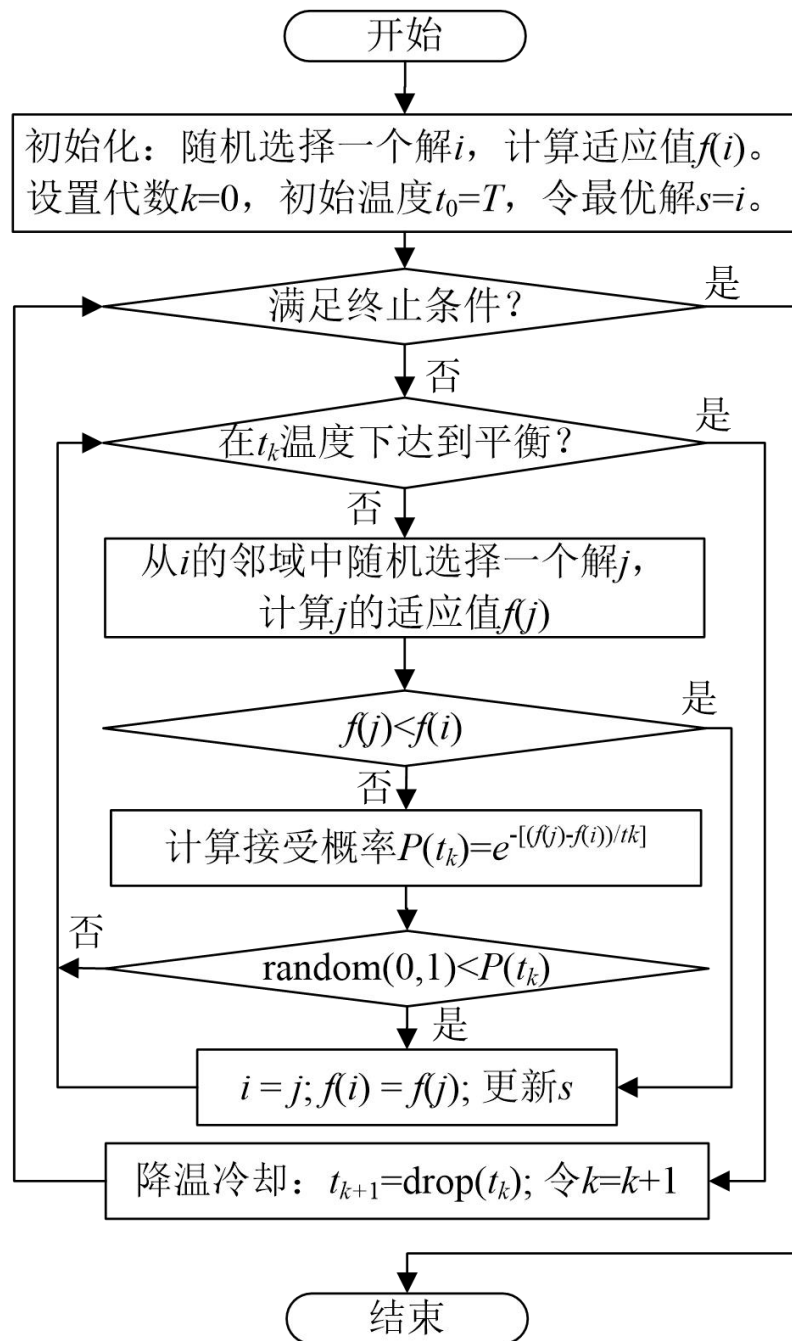
类比关系

模拟退火算法

- 问题的解空间
- 解的质量
- 控制参数
- 设定初始温度
- 控制参数的修改
- 解在邻域中的变化
- 最优解

模拟退火算法

模拟退火基本流程



//功能: 模拟退火算法伪代码

//说明: 本例以求问题最小值为目标

//参数: T 为初始温度; L 为内层循环次数

procedure SA

//Initialization

Randomly generate a solution X_0 , and calculate its fitness value $f(X_0)$;

$X_{best} = X_0; k=0; t_k=T;$

while not stop

//The search loop under the temperature t_k

for $i=1$ to L //The loop times

Generate a new solution X_{new} based on the current solution X_k , and calculate its fitness value $f(X_{new})$.

if $f(X_{new}) < f(X_k)$

$X_k = X_{new};$

if $f(X_k) < f(X_{best})$ $X_{best} = X_k;$

continues;

end if

Calculate $P(t_k) = e^{-(f(X_{new})-f(X_k))/tk};$

if $\text{random}(0,1) < P$

$X_k = X_{new};$

end if

end for

//Drop down the temperature

$t_{k+1} = \text{drop}(t_k); k=k+1;$

end while

print X_{best}

end procedure

功能意义

基本要素

设置方法

影响模拟退火算法全局搜索性能的重要因素之一。
实验表明，初温越大，获得高质量解的几率越大，但花费的计算时间将增加。

初始温度

- 1、均匀抽样一组状态，以各状态目标值的方差定初温
- 2、随机产生一组状态，以两两状态间最大差值定初温
- 3、利用经验公式给出初温

状态空间与状态产生函数。
邻域函数（状态产生函数）应尽可能保证产生的候选解遍布全部解空间。

邻域函数

候选解一般采用按照某一概率密度函数对解空间进行随机采样来获得。
概率分布可以是均匀分布、正态分布、指数分布等等

指从一个状态 X_k （一个可行解）向另一个状态 X_{new} （另一个可行解）的转移概率，通俗的理解是接受一个新解为当前解的概率

接受概率

一般采用Metropolis准则

$$P_{ij}^T = \begin{cases} 1, & \text{if } E(j) \leq E(i) \\ e^{-\frac{E(j)-E(i)}{KT}} = e^{-\frac{\Delta E}{KT}}, & \text{otherwise} \end{cases}$$

指从某一较高温状态 t_0 向较低温状态冷却时的降温管理表，或者说降温方式

冷却控制

- 1、经典模拟退火算法的降温方式 $t_k = \frac{t_0}{\lg(1+k)}$
- 2、快速模拟退火算法的降温方式 $t_k = \frac{t_0}{1+k}$

内层平衡也称Metropolis抽样稳定准则，用于决定在各温度下产生候选解的数目

内层平衡

- 1、检验目标函数的均值是否稳定
- 2、连续若干步的目标值变化较小
- 3、预先设定的抽样数目，内循环代数

算法的终止条件

终止条件

- 1、设置终止温度的阈值
- 2、设置外循环迭代次数
- 3、算法搜索到的最优值连续若干步保持不变
- 4、检验系统熵是否稳定

模拟退火应用举例（0-1背包问题）

- ✎ 假定所有物品的重量和价格都是非负的。背包所能承受的最大重量为 W 。如果限定每种物品只能选择0个或1个，则问题称为0-1背包问题。
- ✎ 例 已知背包的装载量为 $c=8$ ，现有 $n=5$ 个物品，它们的重量和价值分别是 $(2, 3, 5, 1, 4)$ 和 $(2, 5, 8, 3, 6)$ 。试使用模拟退火算法求解该背包问题，写出关键的步骤。
- ✎ 求解：假设问题的一个可行解用0和1的序列表示，例如 $i=(1010)$ 表示选择第1和第3个物品，而不选择第2和第4个物品。用模拟退火算法求解的关键过程如图所示：

运行步骤

已知:

物体个数: $n=5$

背包容量: $c=8$

重量 $w = (2, 3, 5, 1, 4)$

价值 $v = (2, 5, 8, 3, 6)$

第一步: 初始化。假设初始解为 $i=(11001)$, 初始温度为 $T=10$ 。计算 $f(i)=2+5+6=13$, 最优解 $s=i$

第三步: 降温, 假设温度降为 $T=9$ 。如果没有达到结束标准, 则返回第二步继续执行

假设在继续运行的时候, 从当前解 $i=(10110)$ 得到一个新解 $j=(00111)$, 这时候的函数值为 $f(j)=8+3+6=17$, 这是一个全局最优解。可见上面过程中接受了劣解是有好处的。

第二步: 在 T 温度下局部搜索, 直到“平衡”, 假设平衡条件为执行了3次内层循环。

(2-1) 产生当前解 i 的一个邻域解 j (如何构造邻域根据具体的问题而定, 这里假设为随机改变某一位的0/1值或者交换某两位的0/1值), 假设 $j=(11100)$

要注意产生的新解的合法性, 要舍弃那些总重量超过背包装载量的非法解

(2-2) $f(j) = 2+5+8=15 > 13=f(i)$, 所以接受新解 $i=j$, $f(i)=f(j)=15$; 而且 $s=i$;
要注意求解的是最大值, 因此适应值越大越优

(2-3) 返回 (2-1) 继续执行。

(a) 假设第二轮得到的新解 $j=(11010)$, 由于 $f(j) = 2+5+3=10 < 15=f(i)$, 所以需要计算接受概率 $P(T)=\exp((f(j)-f(i))/T) = \exp(-0.5) = 0.607$, 假设 $\text{random}(0,1) > P(T)$, 则不接受新解

(b) 假设第三轮得到的新解 $j=(10110)$, 由于 $f(j) = 2+8+3=13 < 15=f(i)$, 所以需要计算接受概率 $P(T)=\exp((f(j)-f(i))/T) = \exp(-0.3) = 0.741$, 假设 $\text{random}(0,1) < P(T)$, 则接受新解
按照一定的概率接受劣解, 也是跳出局部最优的一种手段

(2-4) 这时候, T 温度下的“平衡”已达到 (即已经完成了3次的邻域产生), 结束内层循环

第二节：禁忌搜索

禁忌搜索算法思想

禁忌搜索算法（Tabu Search, TS）
是 Glover 于 1986 年提出的一种全局搜索算法。



禁忌搜索是属于模拟人类智能的一种优化算法，
它模仿了人类的记忆功能，在求解问题的过程中，
采用了禁忌技术，对已经搜索过的局部最优解进行标记，
并且在迭代中尽量避免重复相同的搜索（但不是完全隔绝），
从而获得更广的搜索区间，
有利于寻找到全局最优解。



禁忌搜索相关概念

∞ 禁忌表 (Tabu List, TL)

∞ 是用来存放（记忆）禁忌对象的表。它是禁忌搜索得以进行的基本前提。禁忌表本身是有容量限制的，它的大小对存放禁忌对象的个数有影响，会影响算法的性能。

∞ 禁忌对象 (Tabu Object, TO)

∞ 是指禁忌表中被禁的那些变化元素。禁忌对象的选择可以根据具体问题而制定。例如在旅行商问题 (Traveling Salesman Problem, TSP) 中可以将交换的城市对作为禁忌对象，也可以将总路径长度作为禁忌对象。

禁忌搜索相关概念

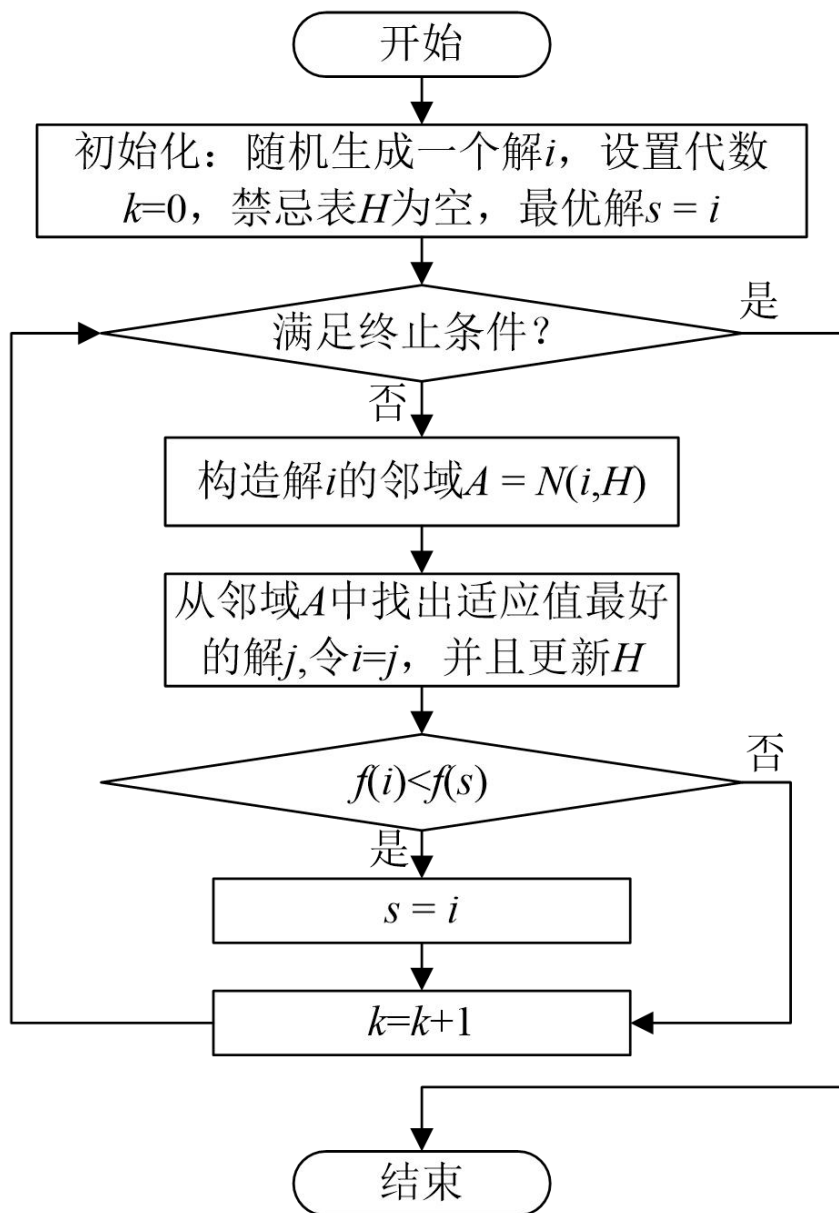
∞ 禁忌期限 (Tabu Tenure, TT)

∞ 也叫禁忌长度，指的是禁忌对象不能被选取的周期。禁忌期限过短容易出现循环，跳不出局部最优，长度过长会造成计算时间过长。

∞ 渴望准则 (Aspiration Criteria, AC)

∞ 也称为特赦规则。当所有的对象都被禁忌之后，可以让其中性能最好的被禁忌对象解禁，或者当某个对象解禁会带来目标值的很大改进时，也可以使用特赦规则。

禁忌搜索基本流程



//功能: 禁忌搜索算法伪代码

//说明: 本例以求问题最小值为目标

//参数: i 为当前解; s 为最优解; H 为禁忌表; k 为
// 当前代数

procedure TS

//Initialization

Randomly generate a solution i , and evaluate it $f(i)$;
 $s=i$; $k=0$; $H=\{\}$;

while not stop

//Generate the neighbors of solution i

// based on the tabu list H

$A = N(i, H)$;

$i = \text{SelectBestSolution}(A)$;

Update the tabu list H ;

if $f(i) < f(s)$

$s=i$;

end if

$k=k+1$;

end while

print s

end procedure

禁忌搜索应用举例

- ✎ 例已知一个旅行商问题为四城市 (a, b, c, d) 问题，城市间的距离如矩阵 $\mathbf{D}$ 所示。
- ✎ 假设邻域映射定义为两个城市位置对换，而始点和终点城市都是 a 。
- ✎ 请分析使用禁忌搜索算法求解该问题的前面三代的过程与主要步骤。

$$\mathbf{D} = d_{ij} = \begin{bmatrix} 0 & 1 & 0.5 & 1 \\ 1 & 0 & 1 & 1 \\ 1.5 & 5 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

禁忌搜索应用举例

- ✧ 从距离矩阵 D 可以看到，这是一个非对称的TSP问题，但是这并不影响算法的执行。
- ✧ 由于题目假设了邻域构造的方式，而且规定了始点和终点都是城市a，因此，在以下的求解过程中，我们不使用城市a和其他城市进行交换，这样的操作并不会影响全局寻优的能力。

运行步骤

$$\mathbf{D} = d_{ij} = \begin{bmatrix} 0 & 1 & 0.5 & 1 \\ 1 & 0 & 1 & 1 \\ 1.5 & 5 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

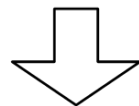
步骤0. 产生初始解 $i = (abcd)$, 设置禁忌表 H 为空, 最优解 $s = i$; 而且根据距离矩阵可以算得 $f(s) = 1+1+1+1=4$



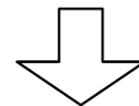
第一代

步骤1-1. 在 H 的限制下构造解 i 的邻域 $A = N(i, H) = \{$

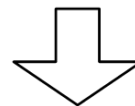
交换 bc
($acbd$),



交换 bd
($adcb$),



交换 cd
($abdc$) }



步骤1-2. 计算邻域 A 中每个新解的适应值

$f=0.5+5+1+1=7.5$ $f=1+1+5+1=8$ $f=1+1+1+1.5=4.5$

选择其中最好的解 j 替换解 i , 所以, $i = j = (abdc)$

更新禁忌表 $H = \{ (cd)2 \}$, 这里的2是禁忌长度, 意思是说未来2代内避免交换城市 c 和 d

运行步骤

$$\mathbf{D} = d_{ij} = \begin{bmatrix} 0 & 1 & 0.5 & 1 \\ 1 & 0 & 1 & 1 \\ 1.5 & 5 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$



第二代

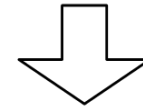
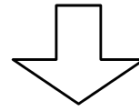
步骤2-1. 在 H 的限制下构造解 i 的邻域 $A = N(i, H) = \{$

交换 bc

$(acdb),$

交换 bd

$(adbc) \quad \}$



步骤2-2. 计算邻域 A 中每个新解的适应值

$$f = 0.5 + 1 + 1 + 1 = 3.5$$

$$f = 1 + 1 + 1 + 1.5 = 4.5$$

选择其中最好的解 j 替换解 i , 所以, $i = j = (acdb)$

更新禁忌表 $H = \{ (cd)1, (bc)2 \}$

步骤2-3. 由于 $f(i) = 3.5 < 4 = f(s)$, 所以最优解 s 更改为当前最优解 $s = i = (ACDB)$, 且 $f(s) = 3.5$

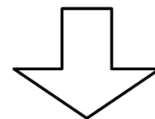
运行步骤

$$\mathbf{D} = d_{ij} = \begin{bmatrix} 0 & 1 & 0.5 & 1 \\ 1 & 0 & 1 & 1 \\ 1.5 & 5 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$



第三代

步骤3-1. 在 H 的限制下构造解 i 的邻域 $A = N(i, H) = \{ \overset{\text{交换}bd}{(acbd)} \}$



步骤3-2. 计算邻域 A 中每个新解的适应值 $f = 0.5 + 5 + 1 + 1 = 7.5$

选择其中最好的解 j 替换解 i ，所以， $i = j = (abdc)$

更新禁忌表 $H = \{ (bc)1, (bd)2 \}$ ，注意禁忌对象 (cd) 现在已经被解禁

步骤3-3. 由于 $f(i) = 7.5 > 3.5 = f(s)$ ，所以最优解 s 保持不变

步骤0. 产生初始解 $i = (abcd)$, 设置禁忌表 H 为空, 最优解 $s = i$; 而且根据距离矩阵可以算得 $f(s) = 1+1+1+1=4$



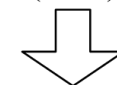
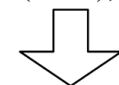
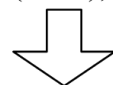
第一代

步骤1-1. 在 H 的限制下构造解 i 的邻域 $A = N(i, H) = \{$

交换 bc
($acbd$),

交换 bd
($adcb$),

交换 cd
($abdc$) $\}$



步骤1-2. 计算邻域 A 中每个新解的适应值 $f = 0.5+5+1+1=7.5$ $f = 1+1+5+1=8$ $f = 1+1+1+1.5=4.5$

选择其中最好的解 j 替换解 i , 所以, $i = j = (abdc)$

更新禁忌表 $H = \{ (cd)2 \}$, 这里的2是禁忌长度, 意思是说未来2代内避免交换城市 c 和 d

步骤1-3. 由于 $f(i) = 4.5 > 4 = f(s)$, 所以最优解 s 保持不变

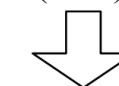
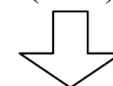


第二代

步骤2-1. 在 H 的限制下构造解 i 的邻域 $A = N(i, H) = \{$

交换 bc
($acdb$),

交换 bd
($adbc$) $\}$



步骤2-2. 计算邻域 A 中每个新解的适应值 $f = 0.5+1+1+1=3.5$ $f = 1+1+1+1.5=4.5$

选择其中最好的解 j 替换解 i , 所以, $i = j = (acdb)$

更新禁忌表 $H = \{ (cd)1, (bc)2 \}$

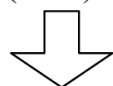
步骤2-3. 由于 $f(i) = 3.5 < 4 = f(s)$, 所以最优解 s 更改为当前最优解 $s = i = (ACDB)$, 且 $f(s) = 3.5$



第三代

步骤3-1. 在 H 的限制下构造解 i 的邻域 $A = N(i, H) = \{$

交换 bd
($acbd$) $\}$



步骤3-2. 计算邻域 A 中每个新解的适应值 $f = 0.5+5+1+1=7.5$

选择其中最好的解 j 替换解 i , 所以, $i = j = (abdc)$

更新禁忌表 $H = \{ (bc)1, (bd)2 \}$, 注意禁忌对象 (cd) 现在已经被解禁

步骤3-3. 由于 $f(i) = 7.5 > 3.5 = f(s)$, 所以最优解 s 保持不变

第三节：免疫算法

免疫算法简介

免疫算法（**Immune Algorithm, IA**）：
是指以在人工免疫系统的理论为基础，
实现了类似于生物免疫系统的抗原识别、细胞分化、
记忆和自我调节的功能的一类算法。



思想来源

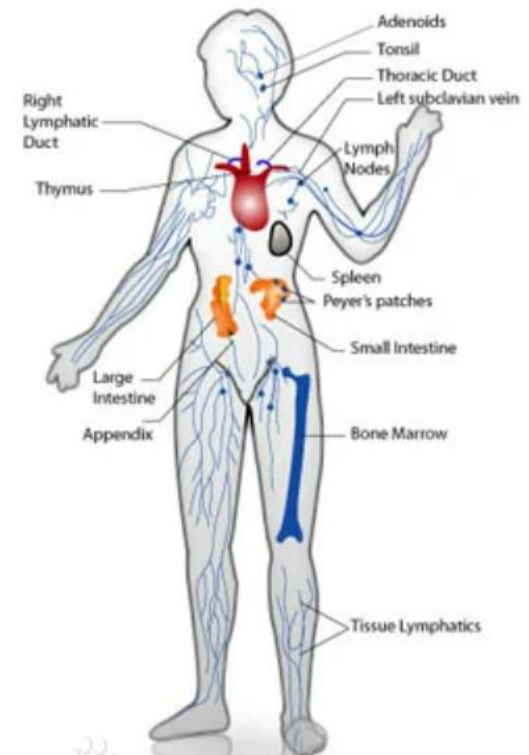
✧ 免疫算法最先起源于1973-1976年间Jernel的三篇关于免疫网络的文章，Jernel在文中提出了一组基于免疫独特型的微分方程，这就是最早的免疫系统。

✧ 免疫算法的主要会议：

International Conference on Artificial
Immune Systems, ICARIS

自然免疫系统

- ❧ 身体拥有很多防御机制，最主要的是皮肤、覆盖内脏和血管的细胞膜，以及自然免疫系统。
- ❧ 自然免疫系统主要由淋巴细胞和淋巴器官组成。这些器官包括扁桃体、腺样增殖体、胸腺、淋巴腺、脾、Peyer 片段、阑尾、淋巴管和骨髓。淋巴器官主要负责免疫系统中淋巴细胞的生长、发展与调配。



自然免疫系统

- ❧ 自然免疫系统会对特定的异体材料或致病物质（指抗原）产生反应。对身体中的正常（自体）和异常（异体或抗原）进行区分。对抗原的识别会导致特定激活细胞的产生，它们会使抗原灭活或摧毁抗原。
- ❧ 自然免疫系统的工作基于模式识别系统，来识别自己模式和异己模式。
- ❧ 在这些反应过程中，自然免疫系统能逐渐自适应调整，从而更好地检测所遇到的抗原，并对经常遇到的抗原产生记忆。由此获得的记忆能够加速自适应免疫系统将来对相同抗原的反应。

自然免疫系统

- ∞ 免疫系统是一个高度进化、复杂的生理机制
- ∞ 免疫系统通过高度复杂的网络结构来识别和排除抗原性异物，维护体内环境的稳定
- ∞ 免疫系统具有识别能力、学习和免疫记忆功能，以及自适应调节机制等特性

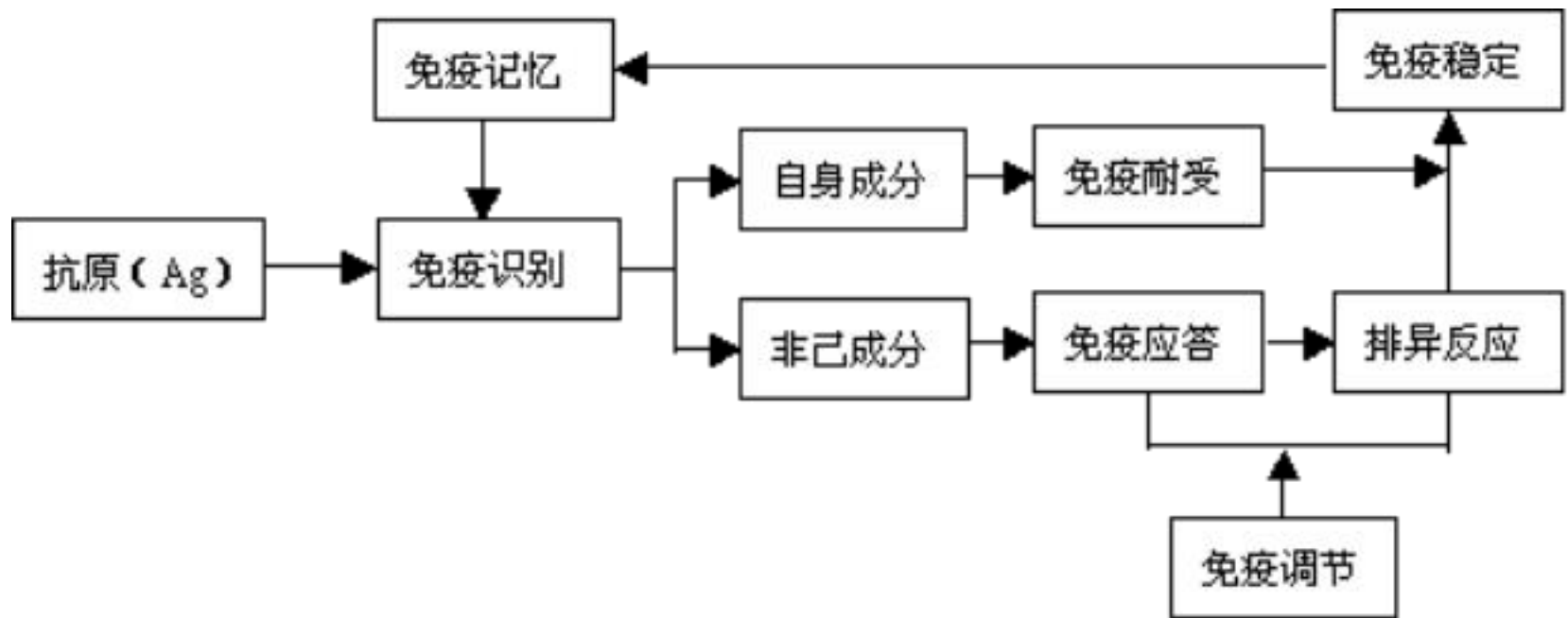


图 3: 免疫反应模型

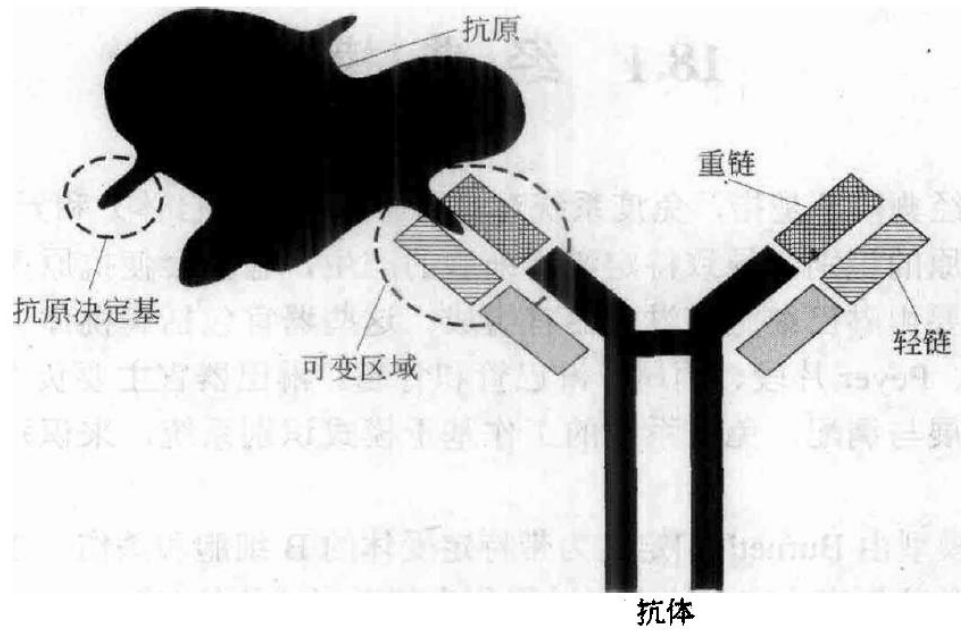
抗原

- ❧ 抗原指的是激发免疫响应的物质。抗原可以是细菌、真菌、寄生虫或病毒。抗原会被识别为异体（异己）。每个细胞在细胞膜表面都有大量的抗原。
- ❧ 抗原表面的小片段称为**抗原决定基**，抗体上面的小片段称为**抗体决定簇**。抗原决定基会激发特定的免疫响应，而抗体决定簇会以一定的强度（称为亲和度）对这些抗原决定基进行绑定。
- ❧ 免疫响应是身体对抗原的反应，从而将抗原清除以防御抗原对身体的破坏。

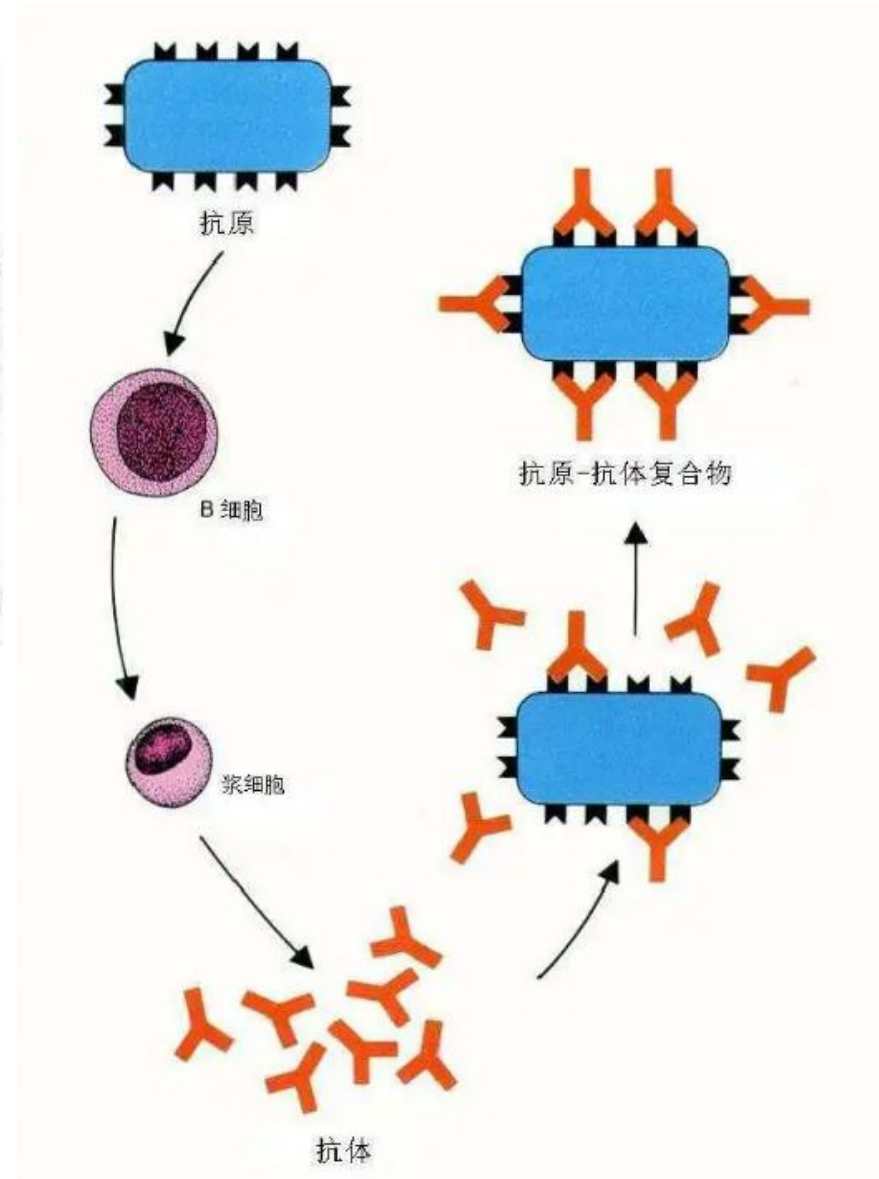
抗体

- ❧ 抗体是化学蛋白质。与抗原相对应，当接触到抗原（异己）时，淋巴细胞会产生抗原细胞，形成部分自体。抗体形状为Y型。Y的两个臂包含两个相同的重链和两个相同的轻链。臂的末端称为可变区域。
- ❧ 可变区域（抗体决定簇）使得抗体能与抗原相匹配，并与抗原的抗原决定基相绑定。当抗体与抗原决定基绑定以后，会形成抗原-抗体-复合体，使得抗原灭活。

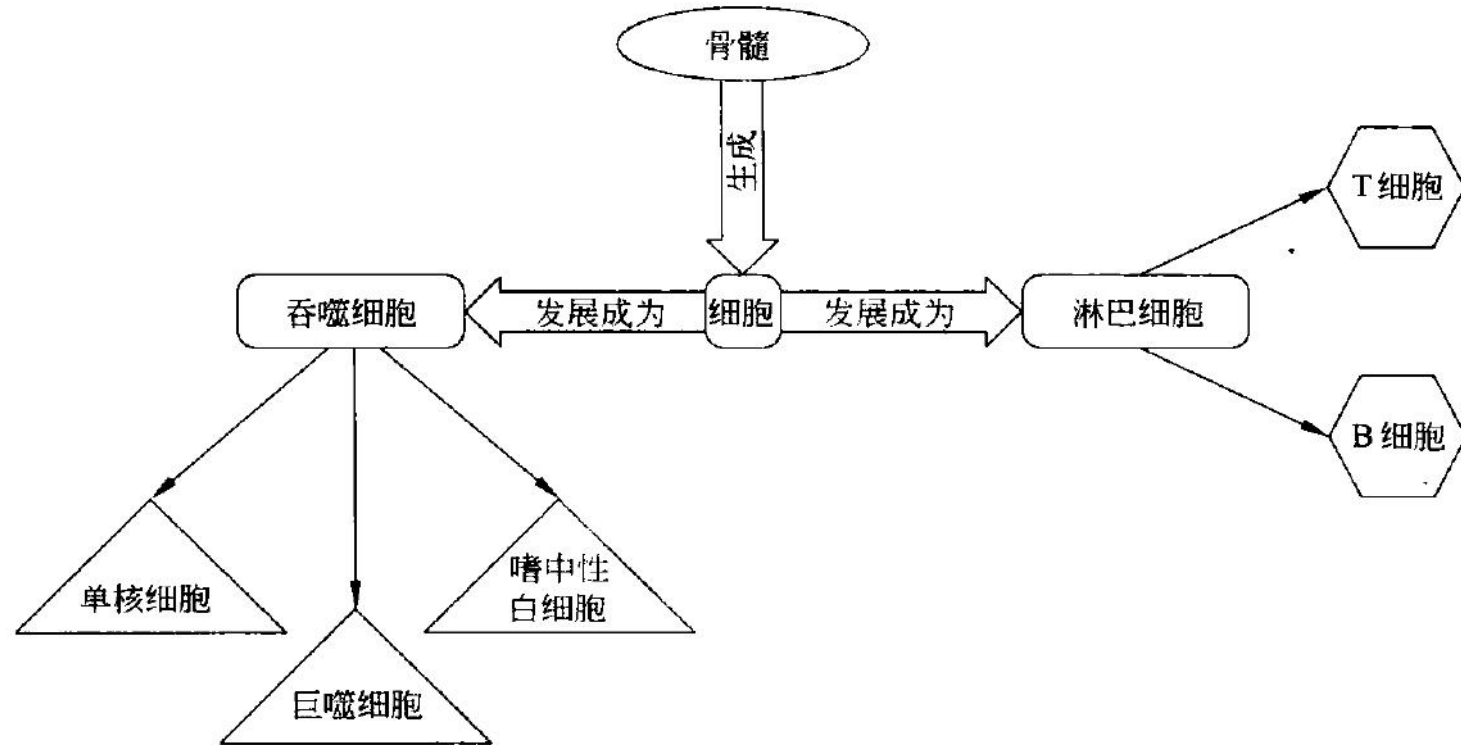
抗原与抗体



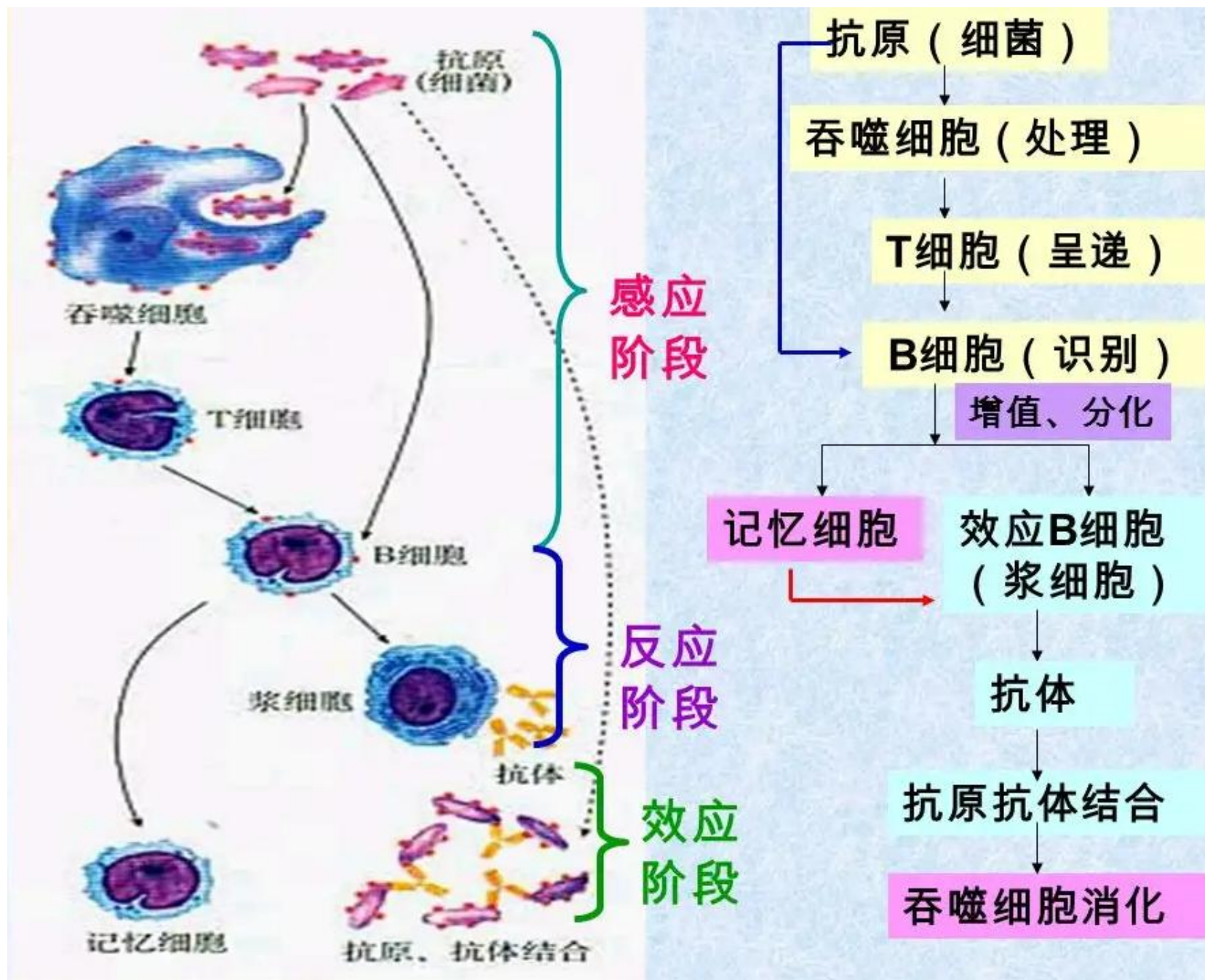
抗原抗体反应



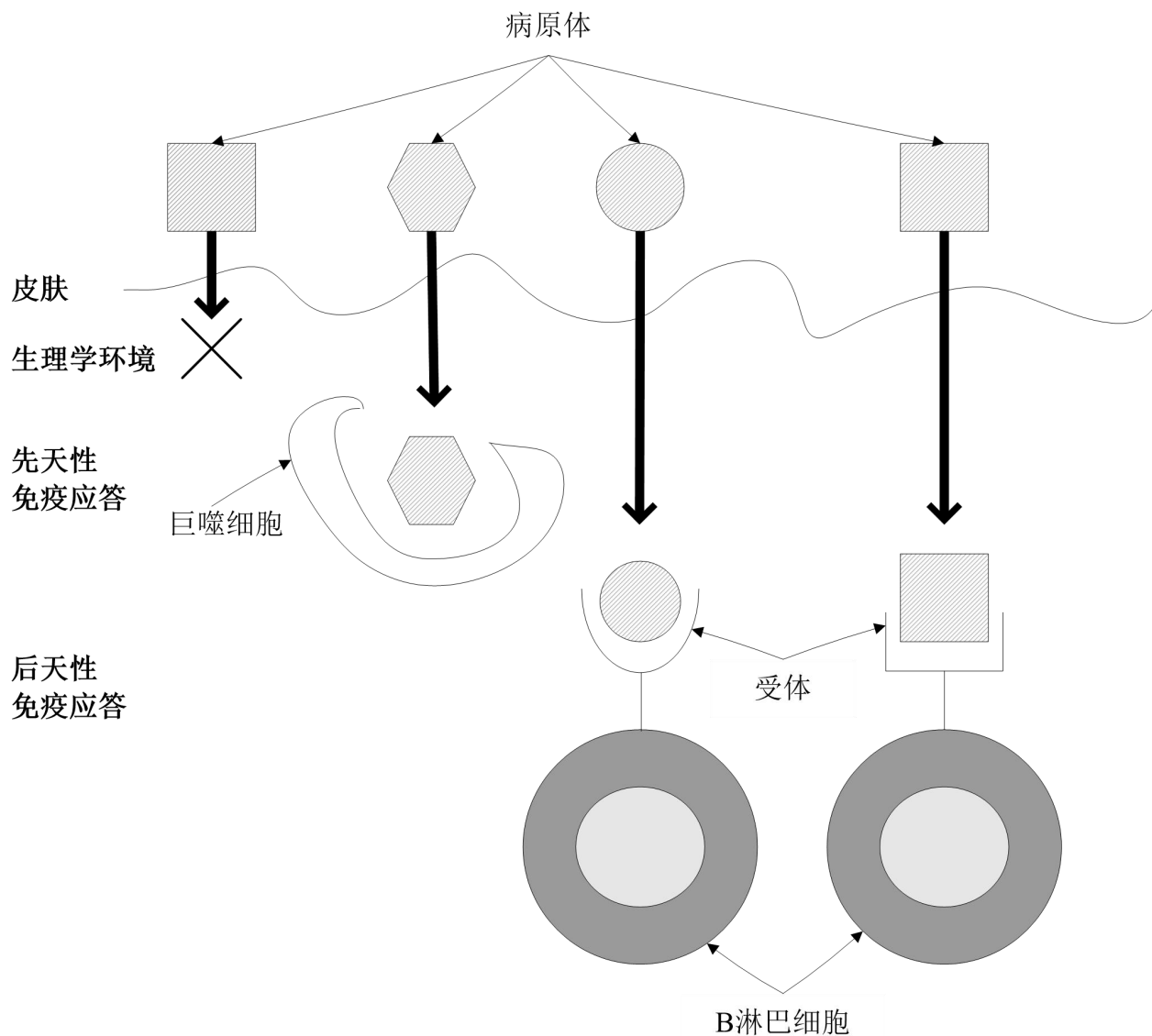
白细胞



- 身体中的所有细胞都是在骨髓中产生的，其中一些细胞发展成大的吞噬白细胞——吞噬细胞。
- 吞噬细胞包括单核细胞、巨噬细胞和嗜中性白细胞。
- 巨噬细胞具有多种功能：分泌功能性化学物质，并在T细胞激活方面有重要作用。
- 其他的细胞发展成为小的白细胞，称为淋巴细胞。



免疫算法的生物模型

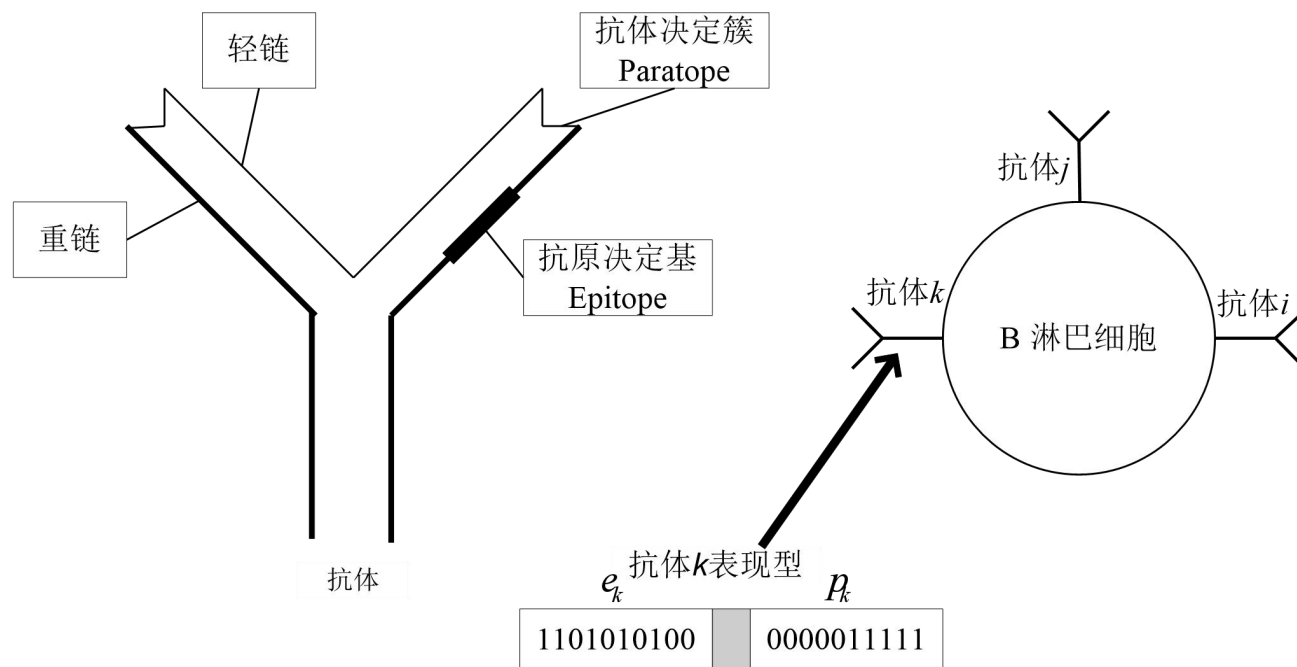


人体免疫系统三个层次示意图

免疫算法的生物模型

- 第一层次是皮肤、粘膜等物理屏障，能够阻挡大部分的病原体。
- 第二层是噬菌细胞，能吞噬病菌并把病菌肢解成小块送到淋巴结。
- 第三层由辅助T细胞（help T cell，白血球的一种）识别病菌，然后告诉人体的淋巴细胞这些入侵敌人的性质，后由淋巴B细胞和T细胞杀死病菌，然后留下B记忆细胞和T记忆细胞把这些病菌的片段（也就是抗原）记忆下来，继续监视病菌的下次入侵。
- 免疫算法所求的问题可以看作是抗原，而所求的解则看作抗体。

二进制模型



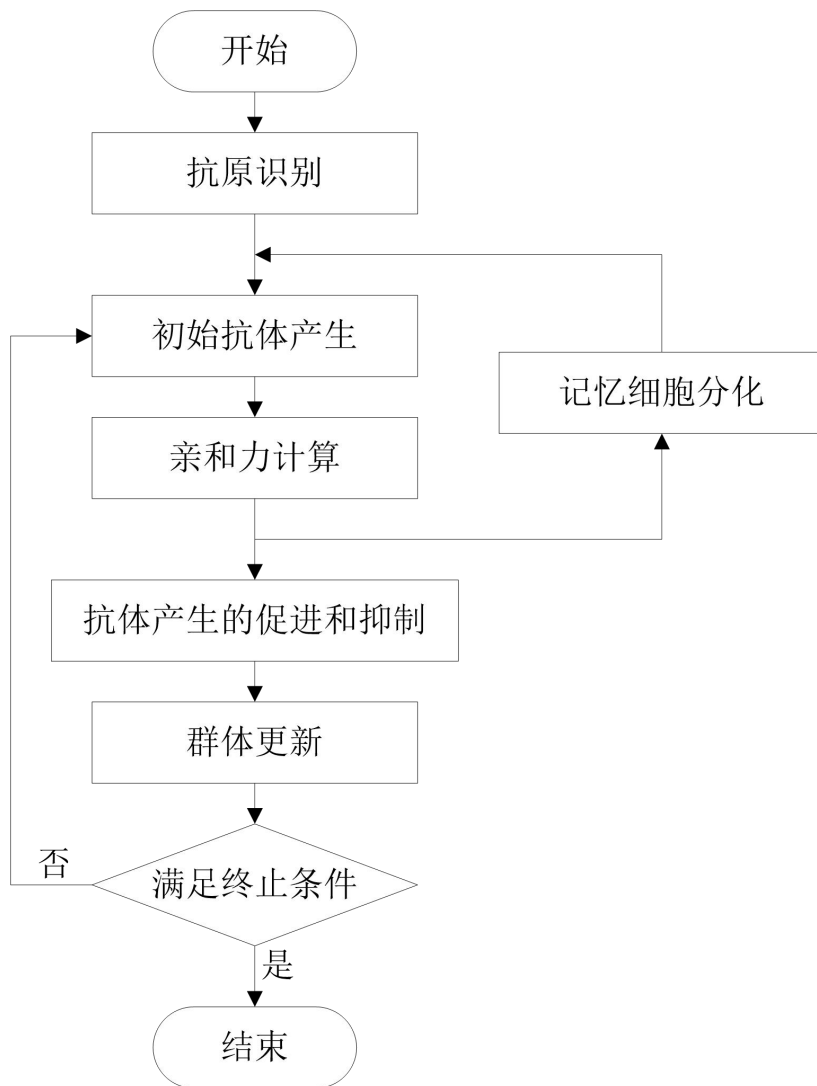
- 左图是一个抗体的缩略图，主要研究抗体决定基和抗原决定簇之间的关系。
- 右图是**B**淋巴细胞的模型，它将是免疫算法主要使用的模型。
- 每个抗体包含两个二进制串，分别表示抗体决定基和抗原决定簇。

免疫算法的基本流程

∞ 免疫系统和免疫算法的比较

免疫系统	免疫算法
抗原	要求解的问题
抗体	最佳解向量
抗原识别	问题识别
从记忆细胞产生抗体	联想过去的成功解
淋巴细胞分化（记忆细胞分化）	维持最优解
T细胞抑制抗体	消除多余的候选解
生命增加（细胞克隆）	用遗传算子生成新的抗体

基本流程



免疫算法的七个要素

- 识别抗体
- 生成初始化的抗体
- 计算亲和度
- 记忆细胞分化
- 抗体促进和抑制
- 产生新的抗体
- 结束条件

计算亲和度

抗体 v 和抗原的亲和度为 ax_v

$$ax_v = \frac{1}{1 + opt_v}$$

其中 opt_v 表示抗体 v 和抗原的结合强度，对最优化问题，可以用抗体 v 的独特型的解和已知的最优解的相似程度表示。

更一般化的基本免疫算法

(1) 求解多目标优化问题的免疫算法

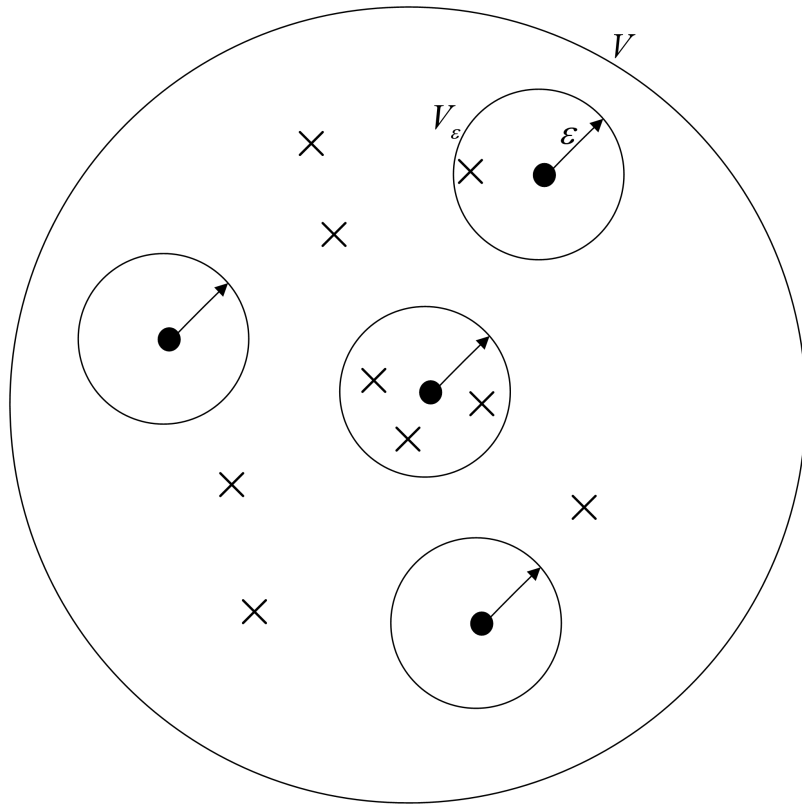
把抗体 v 和抗原 w 的亲合度 $ax_{v,w}$ 重新定义为

$$ax_{v,w} = \frac{1}{1 + opt_{v,w}}$$

其中 $opt_{v,w}$ 表示抗体 v 和抗原 w 的结合强度，即抗体 v 在目标函数 w 的解和此函数最优解的接近程度，

更一般化的基本免疫算法

(2) 求解更一般问题的免疫算法



- 为了求解更多的问题，需要对抗原的基因型进行和抗体一样的编码。
- 免疫系统的形态空间模型，黑圆点表示抗原，交叉表示抗体，同理可以求出抗体之间的距离。
- 然后只需改变亲和度的计算公式，就可以扩展到求一般问题的免疫算法。

更一般化的基本免疫算法

(2) 求解更一般问题的免疫算法

✧ 假设在形态空间内，抗体 v 和抗原的坐标分别为和 g ， $v=1, \dots, N$ ，那么它们之间的距离为

◆ Manhattan距离
$$D = \sqrt{\sum_{i=1}^M |ab_v^i - ag^i|}$$

◆ Euclidean距离
$$D = \sqrt{\sum_{i=1}^M (ab_v^i - ag^i)^2}$$

◆ Hamming距离
$$D = \sum_{i=1}^M \delta_i, \delta_i = \begin{cases} 1, & \text{if } ab_v^i \neq ag^i \\ 0, & \text{otherwise} \end{cases}$$

常用免疫算法

- ④ 负选择算法
- ④ 克隆选择算法
- ④ 免疫算法与智能计算

负选择算法

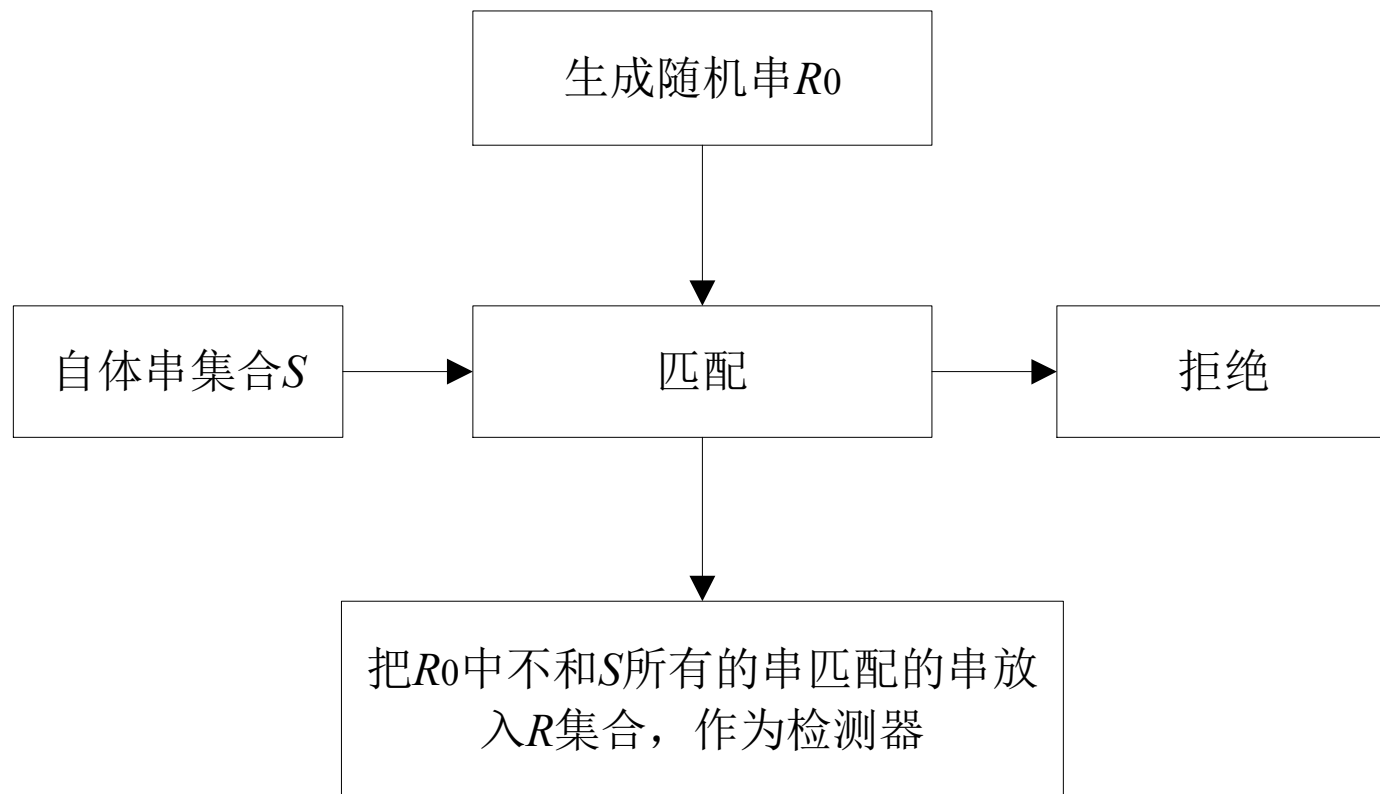
- 1994年，Forrest根据免疫系统的自体-非自体识别原理，提出了负选择算法（Negative Selection Algorithm, NSA）。
- 免疫系统能识别非自体依靠的是T细胞表面的受体，这些受体和所有的自体（身体的器官，组织和细胞）都是不相匹配的，如果接收器和某个蛋白质分子匹配，即可以认为它是非自体，并消灭之。

负选择算法

- ✧ 算法基本思想：需要两个字符串组成的集合 R ，通过先求一个和 S 不匹配的 R 集合，然后用 R 集合判断 S 集合是否发生了变化。
- ✧ 算法分成两部分，第一步是初始化 R ，第二步监视保护数据 S 。

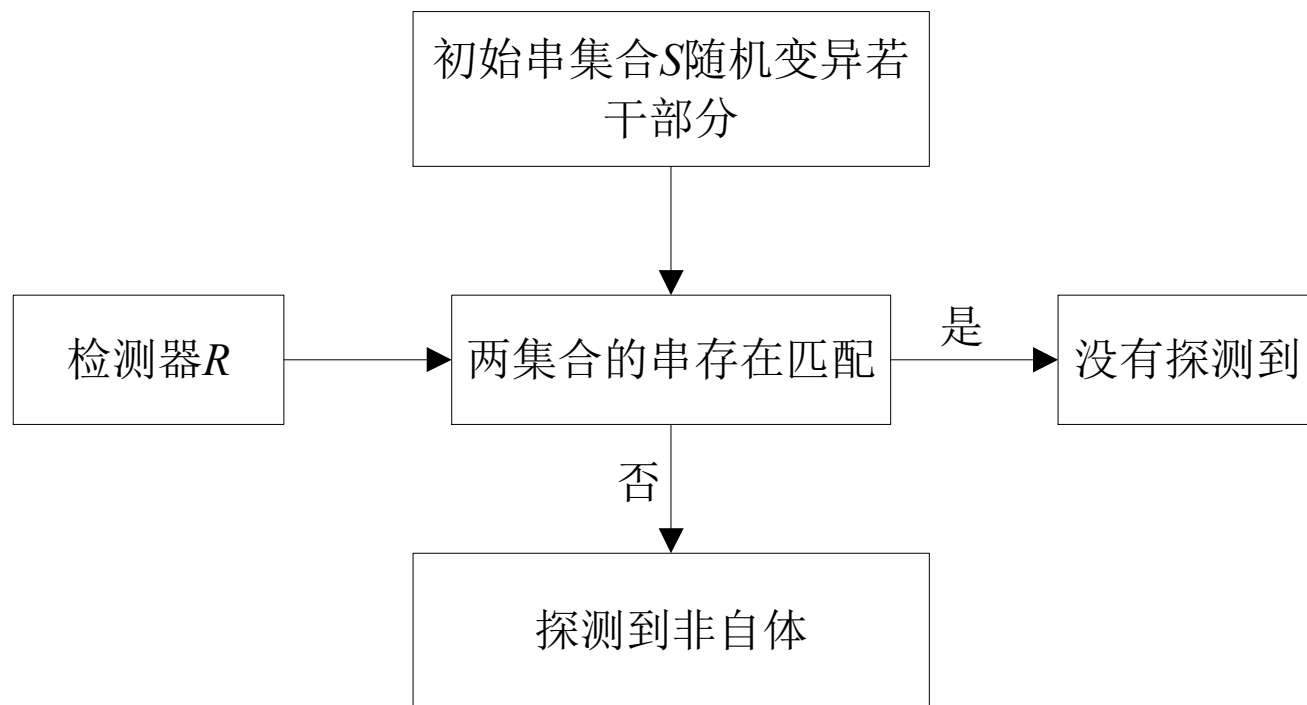
负选择算法

∞ 初始化监测器R



负选择算法

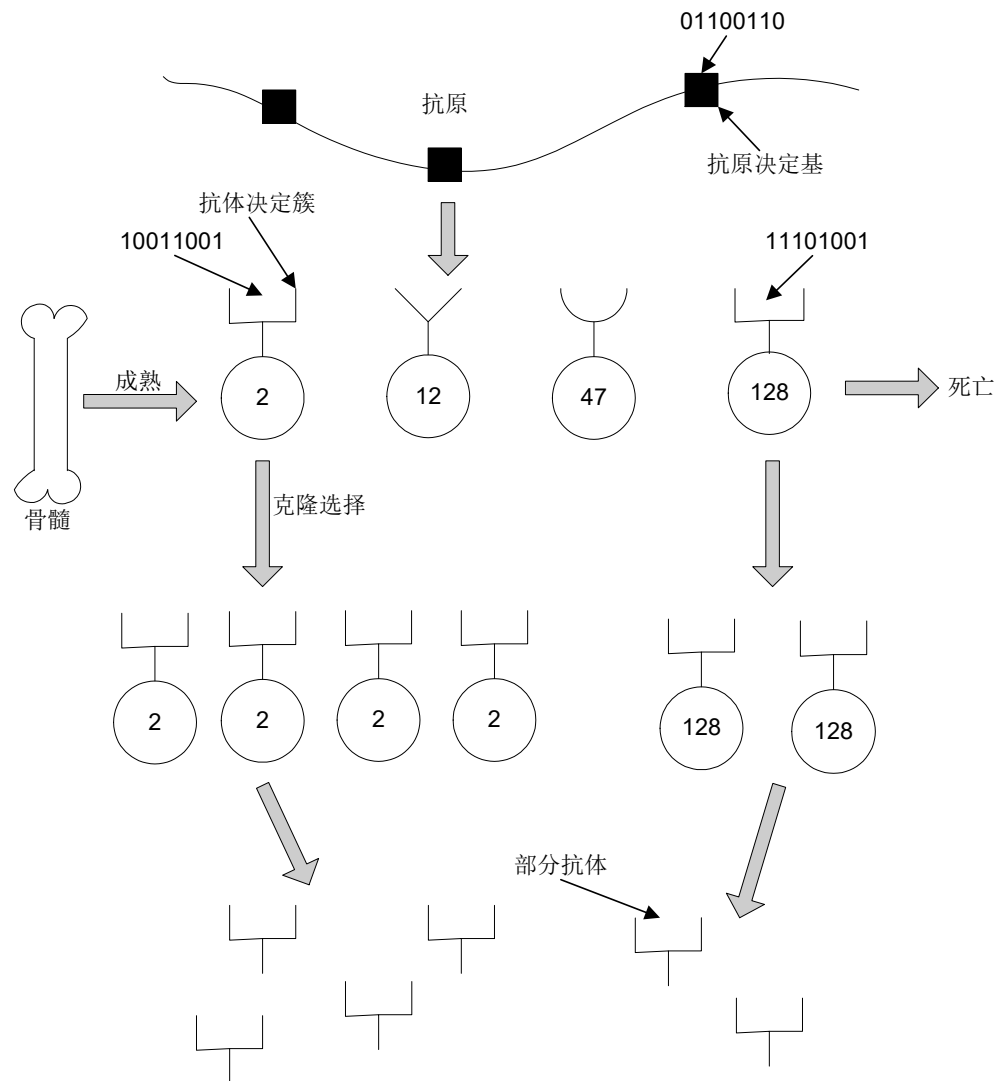
∞ 监视保护数据S



克隆选择算法

∞ 克隆选择原理图

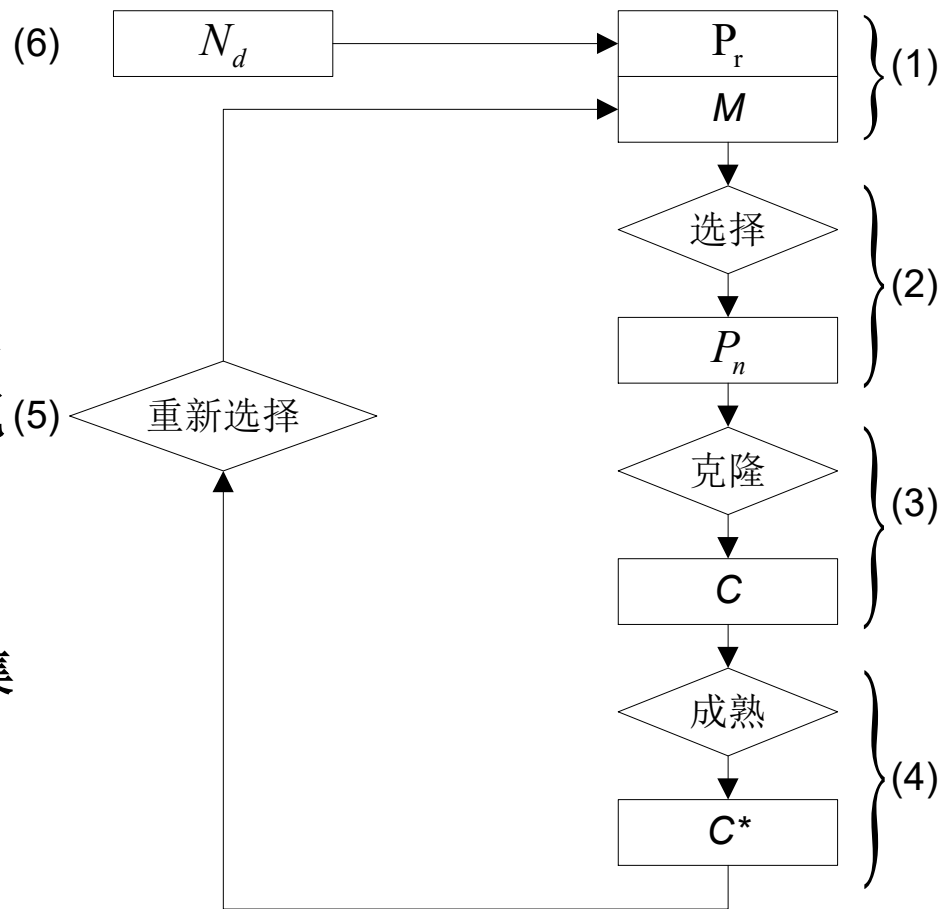
- 抗原的抗原决定基的表现型为{1110011}，在抗体中仅有2号和它完美匹配，128号是部分匹配，所以骨髓刺激2号抗体大量繁殖，128号抗体少量地繁殖，而其他的抗体则不再分裂，等待细胞的死亡。
- De Castro提出了克隆选择算法，他只关注抗体和抗原的亲和度对B细胞的复制的影响，而不考虑抗体之间的亲和度。



克隆选择算法

∞ 克隆选择流程图

- (1) 产生候选解集 P ，它由记忆细胞子集 M 和剩余群体 P_r 组成，即 $P=P_r+M$;
- (2) 根据亲和度的计算，从 P 中找出 n 个最好的个体组成集合 P_n ;
- (3) 克隆 P_n 中的所有 n 个个体，产生一个临时的克隆群体 C ，这个群体 C 的规模是抗原亲和度度量的递增函数;
- (4) 群体 C 经过与抗原亲和度成比例的高频变异，形成一个成熟的群体 C^* ;
- (5) 在 C^* 中重选择改进个体组成记忆集合。 P 集合中的一些成员能够被 C^* 的其他成员所取代;
- (6) 取代群体 P 中 d 个亲和度最低的抗体，维持多样性。



免疫算法与进化计算

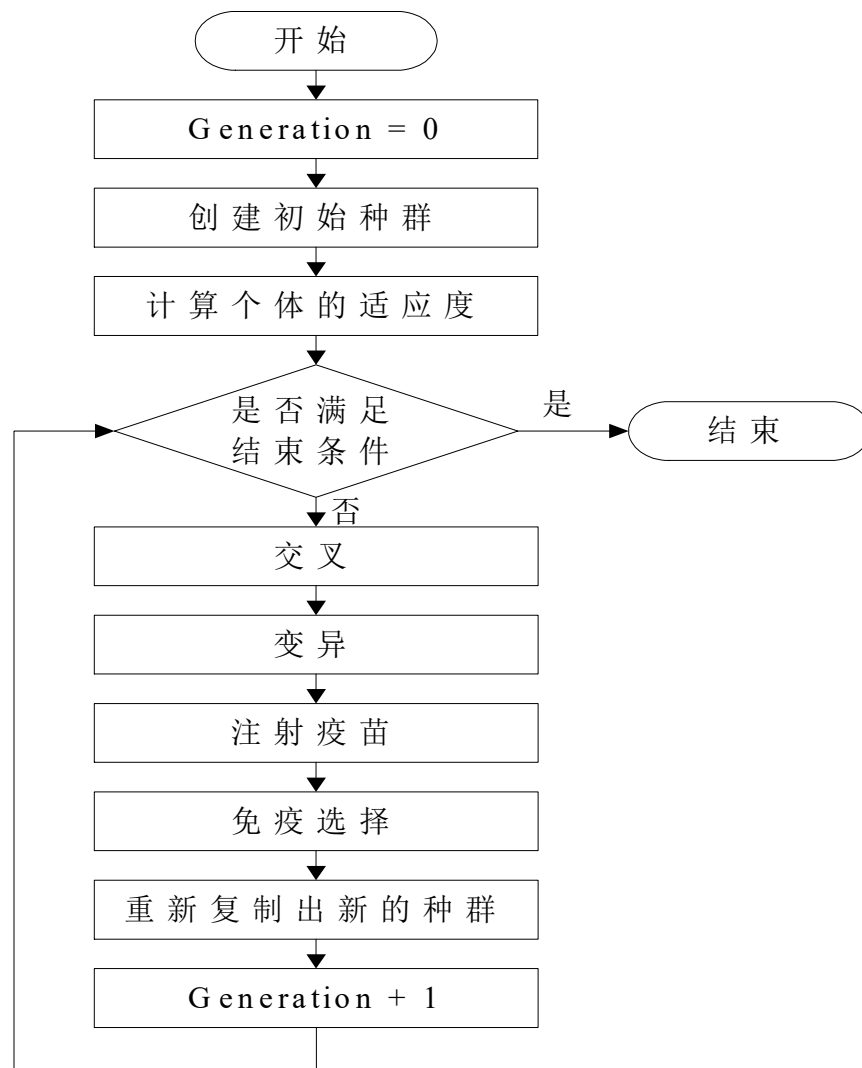
免疫遗传算法

免疫遗传算法IGA和GA最大的不同之处在于两个地方

(1)注射疫苗：所谓疫苗就是一个个体随机变异若干位得到的变异体，是根据先验知识得到的，有较高的概率得到更高的适应度。

(2)免疫选择：

- 第一步称为免疫测试，如果新的个体适应度不如老的个体，还是用老的个体参加后面的运算，否则就用新的个体；
- 第二步称为退火选择，在当前的后代群体中根据轮盘赌的思想选择一个个体 $x_i (i=1, \dots, n)$ 放入新的父种群 A_{k+1} 中



免疫算法的应用

- ∞ 识别与分类问题
- ∞ 优化问题
- ∞ 机器人学习与控制
- ∞ 数据挖掘

免疫算法的应用

- ❧ 早期的免疫系统模型是为了识别电脑病毒而设计出来的，由于免疫系统强大的识别功能，在识别问题上免疫算法得到了广泛的运用。
- ❧ 模式识别是通过计算机用数学技术的方法来研究模式的自动处理和判读，简单地说模式识别就是指导计算机像人类一样的具有自动的对环境和客体的感知能力。
- ❧ 识别的一个基本问题就是区分自体和非自体，这也是免疫系统消灭病原体的一个必然的过程，所以免疫算法也成为模式识别领域的研究热点。

免疫算法的应用

- ✧ 由于淋巴细胞具有很强的学习和记忆功能，免疫算法应用于优化和设计上的也不少。
- ✧ 调度（Scheduling）和规划（Planning）是一类密切影响着我们日常生活的优化问题，例如会议安排、公车路线规划、飞机调度等等。
- ✧ CSA、IGA等免疫算法已经在众多的调度与规划问题上取得了非常成功的应用。从数学的角度讲这些调度，规划问题属于组合优化问题的范畴，都是优化问题。

第四节：Memetic算法

Memetic算法的基本思想

∞ Meme概念的提出—Hawkins（1976）

	基因（Gene）	文化基因（Meme）
相似点	在遗传过程中通过交叉、变异等操作不断演化发展	在传播过程中通过交互、融合、变异等方式传承发展
不同点	在生物进化过程中，变异是随机的，只有少数好的变异能够在自然选择中得到保留	文化传播过程往往都以充分的专业领域知识为基础，进化速度更快

∞ Mosato（1989）提出了Memetic算法

∞ Memetic算法—基于群体的计算智能方法与局部搜索相结合的一类算法的总称

Memetic算法的框架

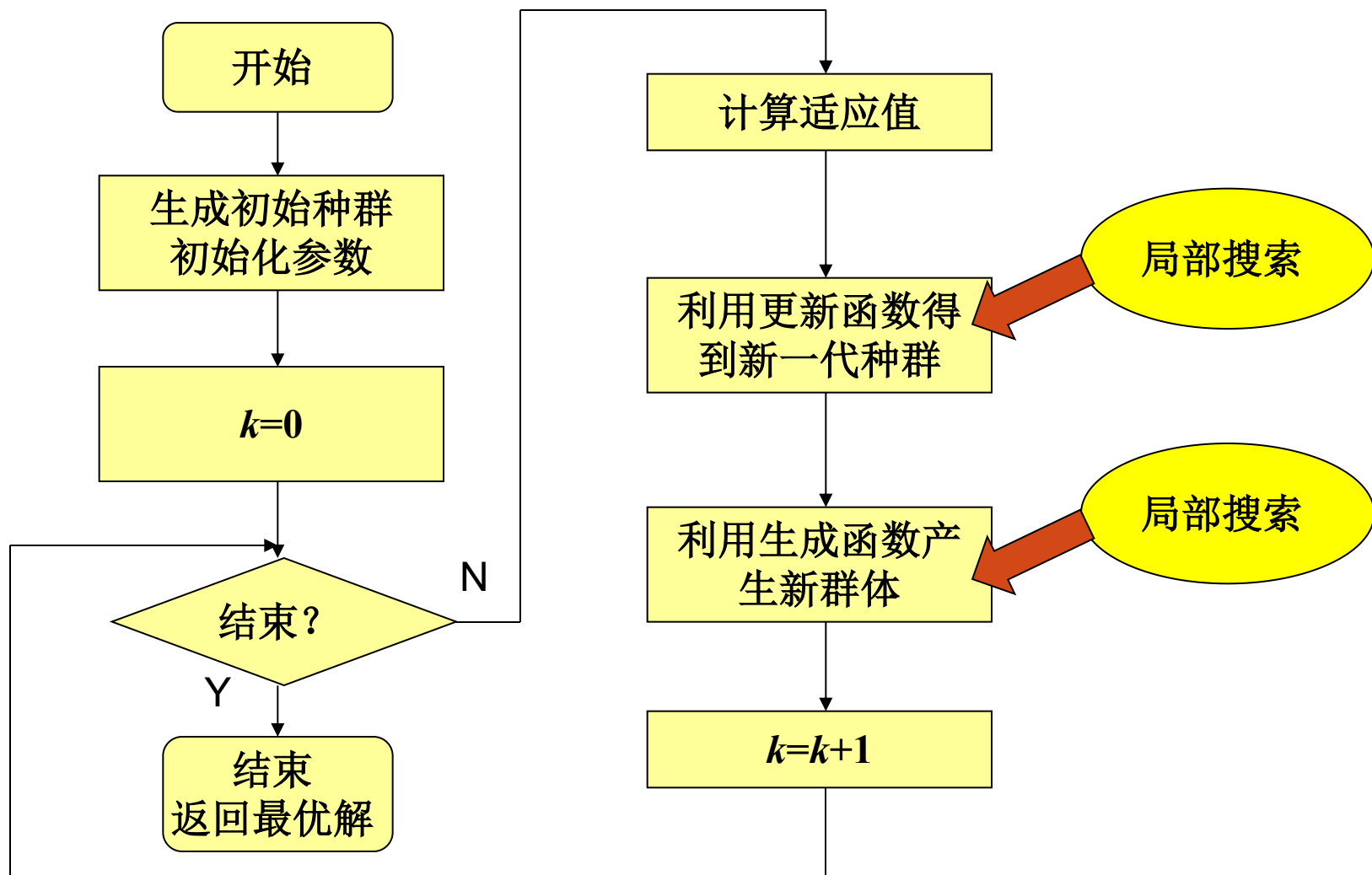
Memetic算法的基本模型（9要素）

$$MA = (P^0, \delta^0, offspringSize, popSize, l, F, G, U, L)$$

$P^0 = (x_1^0, x_2^0, \dots, x_{popSize}^0)$	初始种群
δ^0	算法的初始参数
$popSize$	种群规模
$offspringSize$	通过生成函数得到的后代的数目
l	编码长度
F	适应值函数
G	生成函数（如交叉、变异等算子）
U	更新函数（如选择算子）
$L = (L_1, L_2, \dots, L_m)$	局部搜索策略的集合

Memetic算法的框架

Memetic算法的流程图



Memetic算法的框架

⌘ Memetic的各种设计方案:

⌘ 全局搜索策略的选择

(进化算法 / ACO / PSO.....)

⌘ 局部搜索策略的选择

(单一局部搜索策略 / 多种局部搜索策略)

⌘ 局部搜索策略的位置

(与生成函数相结合 / 与更新函数相结合)

⌘ 局部搜索的方式 (Lamarckian / Baldwinian)

⌘ 局部搜索的对象 (整个群体 / 部分个体)

⌘ 局部搜索与全局搜索的平衡

(局部搜索的强度与频率)

⌘ 局部搜索的参数 (邻域的形状和大小 / 移动步长)

静态Memetic算法

∞ 静态Memetic算法的特点

∞ 一般只采用一种局部搜索策略

∞ 局部搜索的执行位置和方式都预先确定，在算法执行过程中保持不变

∞ 目前已提出的Memetic算法，大部分都属于静态Memetic算法

静态Memetic算法

部分代表性的静态Memetic算法

局部搜索的位置	与生成函数相结合	与更新函数相结合
代表性算法	Nguyen et al. 2007 Nguyen et al. 2002 Burke et al. 2000 Sheng et al. 2007 Noman and Iba 2008 等等	Tu and Lu 2004
局部搜索的方式	Lamarckian	Baldwinian
代表性算法	Krasnogor and Smith 2006 Dorigo et al. 2004 ● 一般而言，Lamarckian方式比Baldwinian方式性能更优	

动态Memetic算法

∞ 动态Memetic算法—动态调整局部搜索策略

∞ 分类

∞ 按照动态调整类型，分成三类

∞ 静态型—按照静态的规则来调整

∞ 适应型—利用在线反馈的信息来调整

∞ 自适应型—将局部搜索设置信息也编码到算法当中一起进化（协同进化的MA）

∞ 按照自适应的层次，分成三类

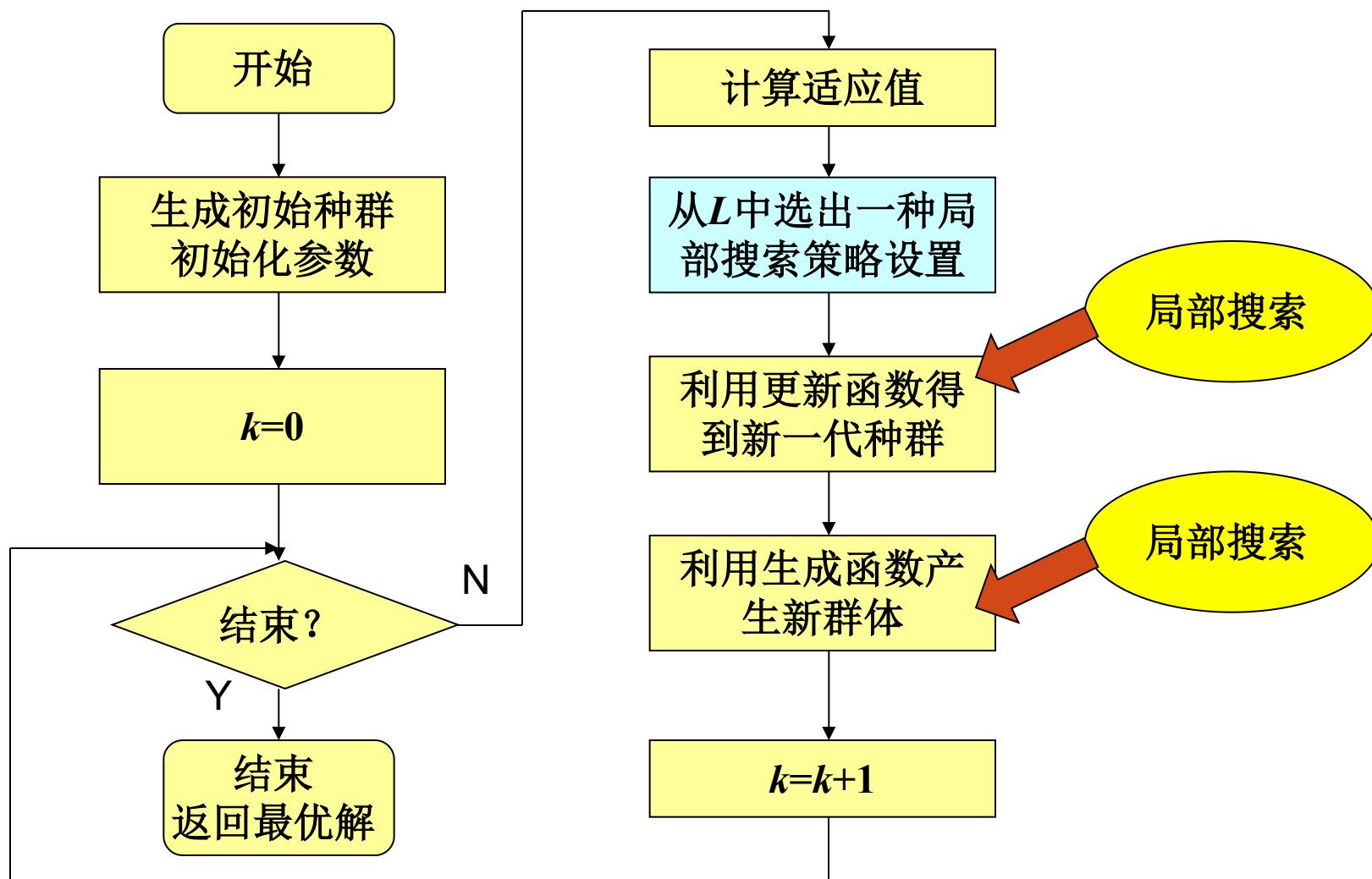
∞ 外部型—利用外部的先验知识（一般属于静态型）

∞ 局部型—采用了局部反馈信息来调整

∞ 自适应型—采用了全部反馈信息来调整

动态Memetic算法

动态Memetic算法的流程图



动态Memetic算法

∞ Meta-Lamarckian学习型Memetic算法

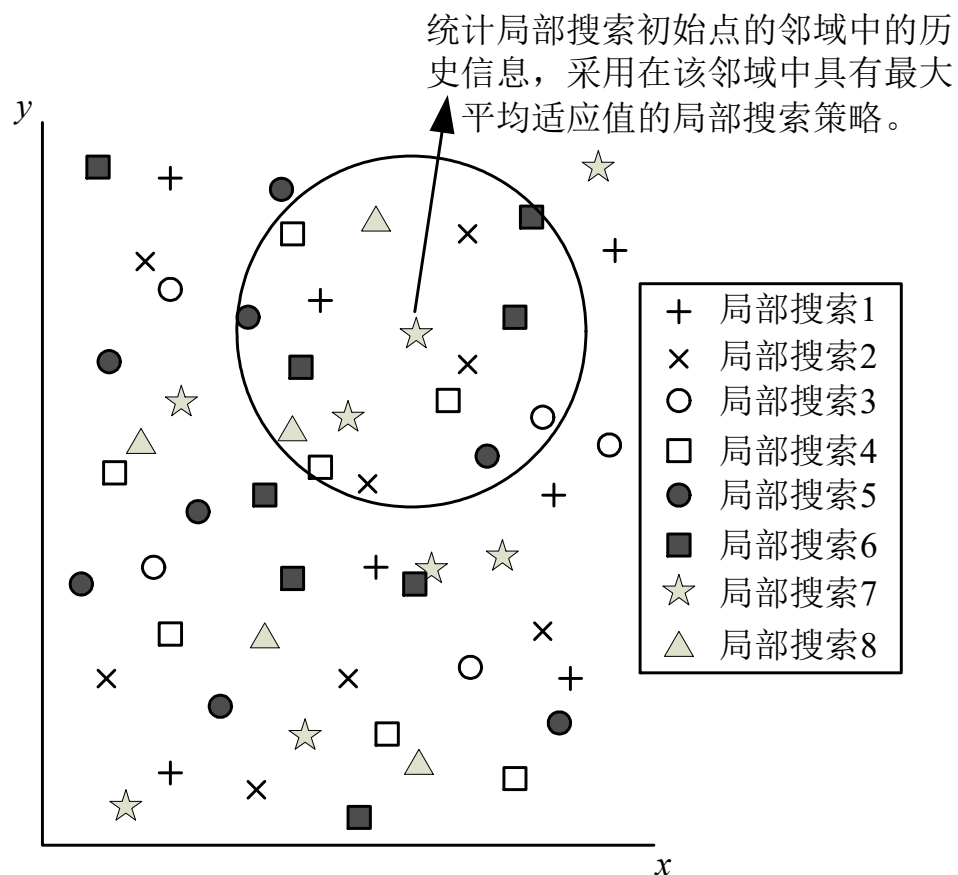
∞ 每次执行局部搜索之前，从局部搜索策略池中选择一种局部搜索方案

∞ 基本Meta-Lamarckian学习方案

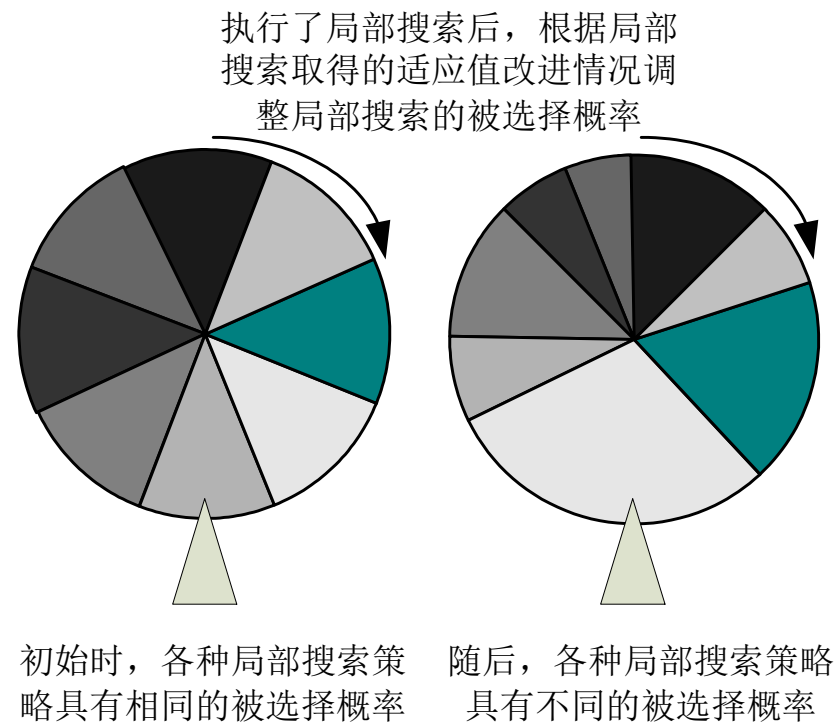
∞ 子问题分解的启发式搜索方案

∞ 带偏向性轮盘赌的随机搜索方案

动态Memetic算法



(a) 子问题分解的启发式搜索方案



(b) 带偏向性轮盘赌的随机搜索方案

动态Memetic算法

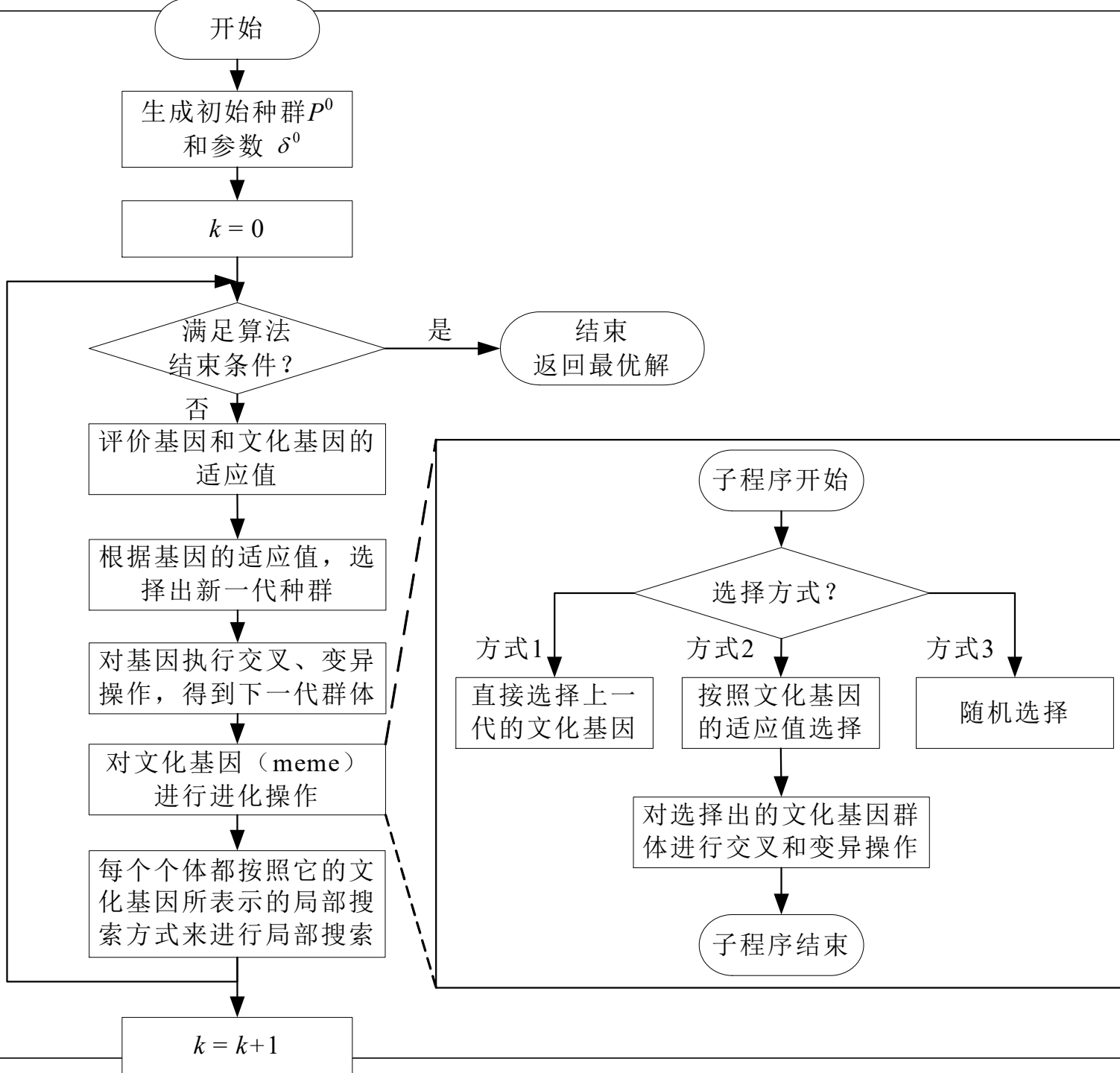
∞ 超启发式Memetic算法

- ∞ 随机超启发式 – 随机地选择局部搜索策略
- ∞ 贪心超启发式 – 选择能够得到最大改进幅度的局部搜索策略
- ∞ 基于选择函数的超启发式 – 根据局部搜索策略的选择函数 F 值来选择局部搜索策略。

∞ 协同进化Memetic算法

局部搜索的具体设置也编码到个体的基因中，共同进化。

协同进化MA的流程图



动态Memetic算法

部分代表性的动态Memetic算法

类型	静态	适应	自适应
外部	基本meta-Lamarckian学习, Simplerandom等		
局部		子问题分解的启发式搜索, 贪心超启发式, Randomdescent, Randomperm descent	协同进化MA (或自生MA), 自适应MA
全局		带偏向性轮盘赌的随机搜索, 基于PSO的MA, 基于选择函数的超启发式, 快速适应MA	

Memetic算法的研究展望

⌘理论研究进展

- ⌘MA的语法框架 (Krasnogor和Smith)

- ⌘MA的收敛性分析 (Ong et al.)

- ⌘MA的搜索行为和收敛性分析还基本处于空白状态

⌘应用研究进展

- ⌘已应用于众多复杂优化问题

⌘应用研究进展

- ⌘自适应与协同进化MA

- ⌘多目标MA

- ⌘基于智能体的MA

第五节：分布估计算法

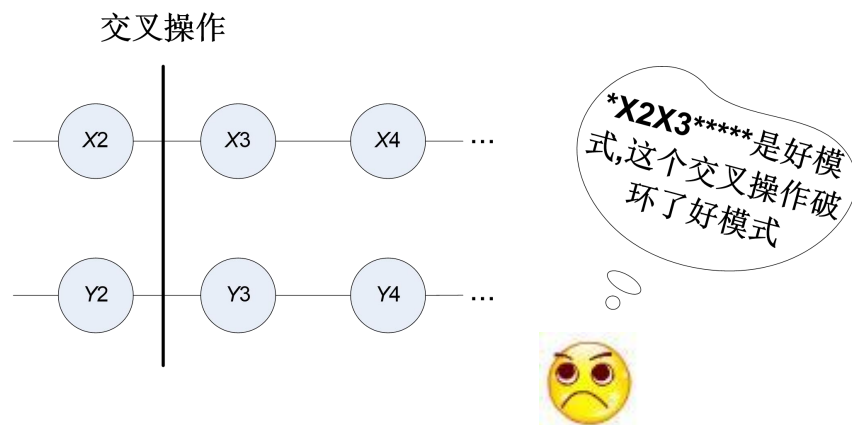
分布估计算法的思想起源

■ 什么是分布估计算法?

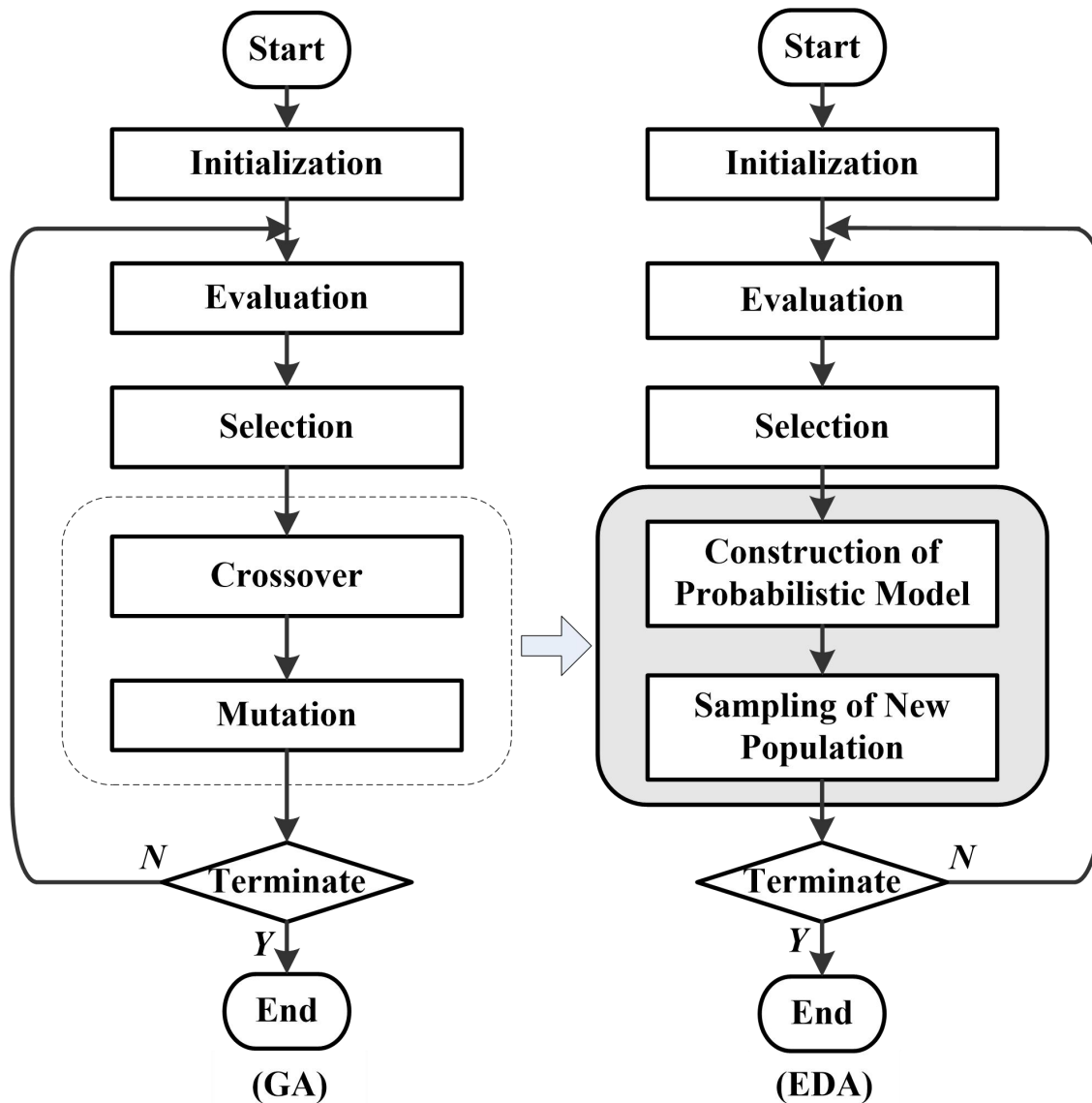
- Estimation of Distribution Algorithm (EDA)
- 基于种群的新型进化算法
- 思想起源于遗传算法

■ 算法的思想起源

- 改进遗传算法的交叉操作和变异操作, 防止破坏好模式.
- 采用概率模型和抽样的隐式形式产生新个体.



分布估计算法与遗传算法的流程比较



分布估计算法的通用流程

Randomly generated the initial population $P(0)$;

$t = 0$;

While not met the termination condition **do**

Begin

 Select a set of promising individuals $D(t)$ form the current population $P(t)$;

 Estimate the probability distribution of the selected set $D(t)$;

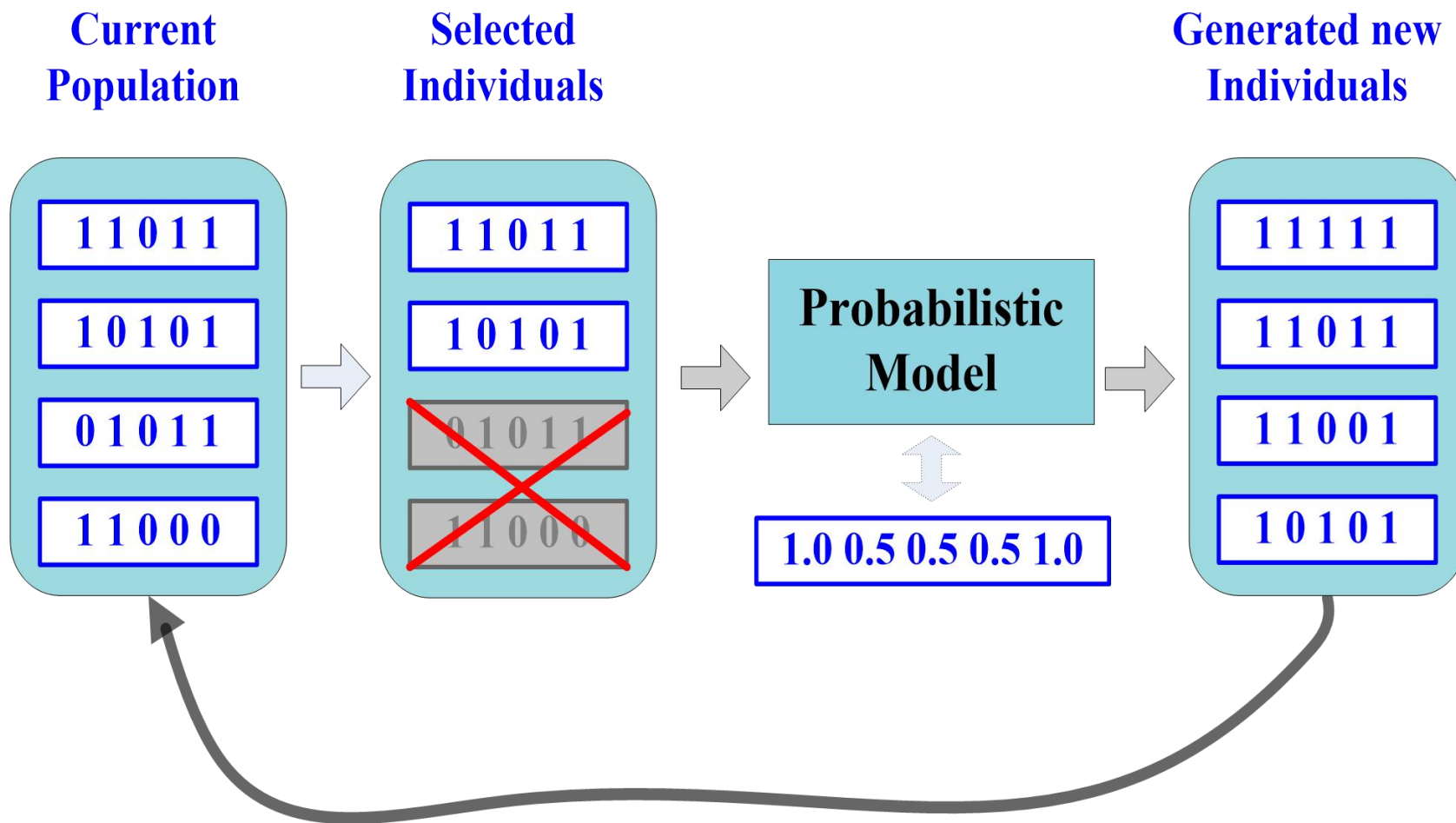
 Generate a set of new individuals $N(t)$ according to the estimate;

 Create a new population $P(t+1)$ by replacing some individuals of $P(t)$ by $N(t)$;

$t = t+1$;

end

一个简单的分布估计算法例子



分布估计算法的发展历史

■ 开山始祖

- **PBIL**:(1994 Baluja)
- **UMDA**: (1996 H.Mihlenbein & Paass)

■ 早期的算法专注于二进制编码

- **MIMIC**: (1997 J.S.D.Bonet)
- **COMIT**:(1997 S.Baluja)
- **FDA**: (1999 HM Uhlenbein)
- **BOA**: (1999 M.Pelikan)

■ 逐渐扩展到连续分布估计算法

- **PBILc** (Sebag, 1998)
- **UMDAc** (Larrañaga, P., et al, 2000)
- **CEDGA** (Q. Lu, 2005)
- **FWH & FHH** (Tsutsui et al , 2001)
- **sur-shr-HEDA**(N. Ding, et al, 2006)

分布估计算法的发展历史

■ 混合分布估计算法

- EDA与粒子群优化的混合
- EDA与遗传算法的混合
- EDA与差分进化算法的混合

■ 并行分布估计算法

- 主从模式
- 岛屿模型

■ 收敛性证明

■ 应用领域越来越广泛

基于高斯模型的EDA

■ 基于高斯模型的EDA的流程如下:

第一步:随机生成初始种群 $P(0)$,初始化高斯模型的均值 μ 与方差 σ . 假设所有变量对应的高斯模型为:

$$(N(\mu_1, \sigma_1), N(\mu_2, \sigma_2), \dots, N(\mu_D, \sigma_D))$$

其中 D 为待求解问题的维数.

第二步:根据各维变量对应的高斯模型,抽样产生新种群 $T(t)$.

第三步:从新种群中选择优秀个体集合 $S(t)$.

第四步:计算 $S(t)$ 在各维变量上的均值与方差,对原有的高斯模型进行更新.

高斯模型的更新方法

■ 更新概率模型中的两个重要参数:均值和方差

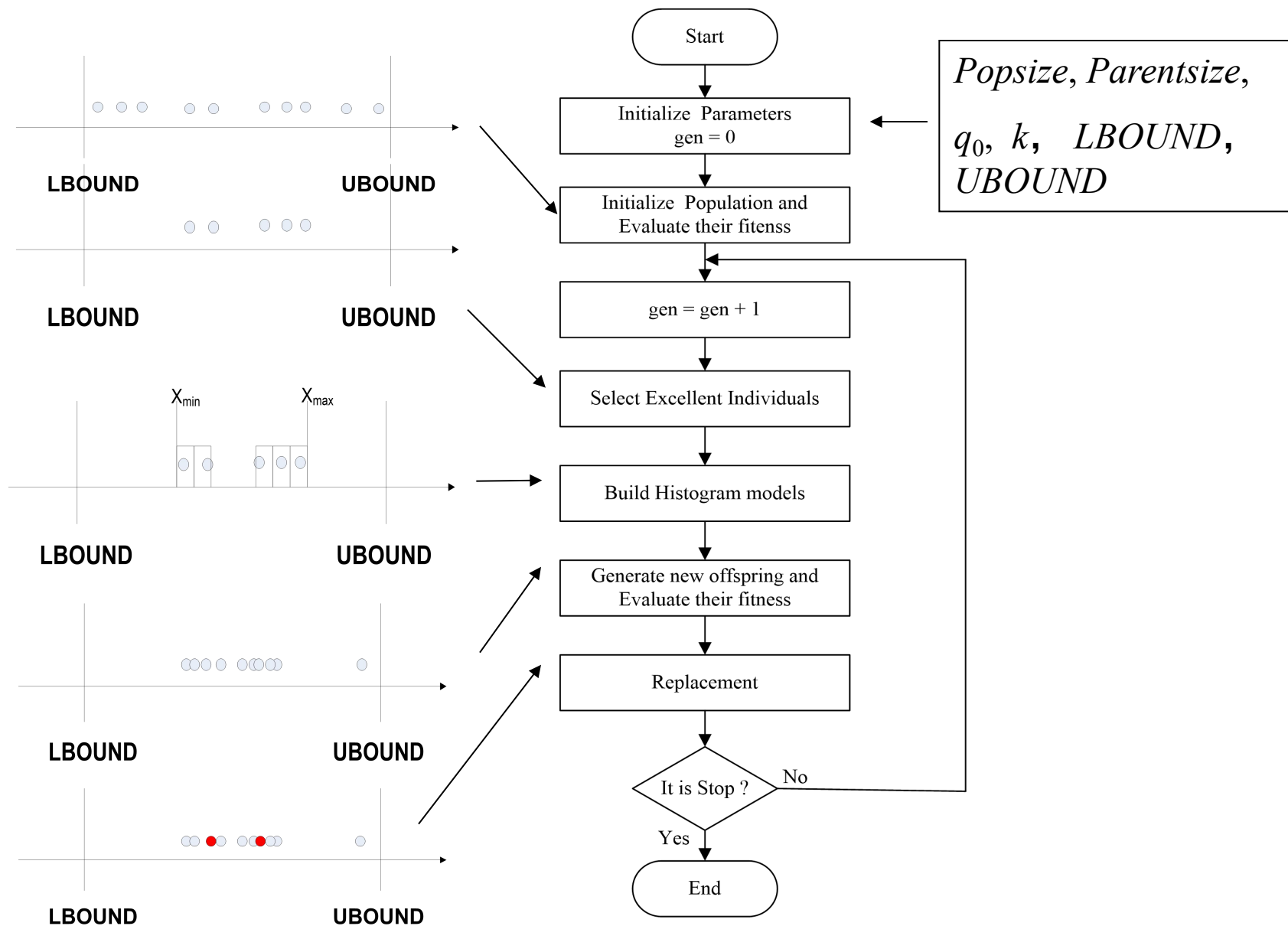
$$\mu_j^{t+1} = (1 - \alpha) \cdot \mu_j^t + \alpha \cdot (x_j^{best,1} + x_j^{best,2} - x_j^{worst})$$

$$\sigma_j^{t+1} = (1 - \alpha) \cdot \sigma_j^t + \alpha \cdot \sqrt{(\sum_{k=0}^{K-1} (x_{jk} - \bar{x})) / K}$$

其中 K 为优秀个体的个数. $x^{best,1}$, $x^{best,2}$ 为最好的两个个体, x^{worst} 则为最差的个体.

更新的方式多种多样,也可以直接用优秀个体的均值和方差替代原来的均值与方差.

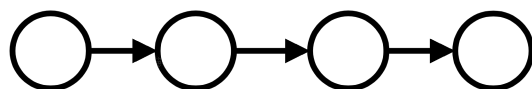
基于直方图概率模型的EDA



基于链式概率模型的EDA

■ Mutual Information Maximization for Input Clustering, MIMIC

- ▣ 变量之间的关系是一种链式的关系，即在 n 维随机变量组成的链中，只有相邻的变量之间才有关系。



- ▣ 联合概率密度在形式上产生变化：

$$p(X) = p(X_1 | X_2 \dots X_n) p(X_2 | X_3 \dots X_n) \dots p(X_{n-1} | X_n) p(X_n)$$

↓

$$\hat{p}_{\pi}(X) = p(X_{i1} | X_{i2}) p(X_{i2} | X_{i3}) \dots p(X_{in-1} | X_{in}) p(X_{in})$$

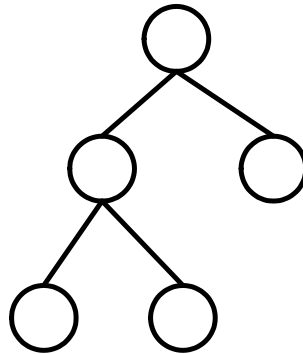
其中 π 是根据链式关系的一种排列。

如何找出最优的 π ？

基于树状概率模型的EDA

■ Combining Optimizers With Mutual Information Trees, COMIT

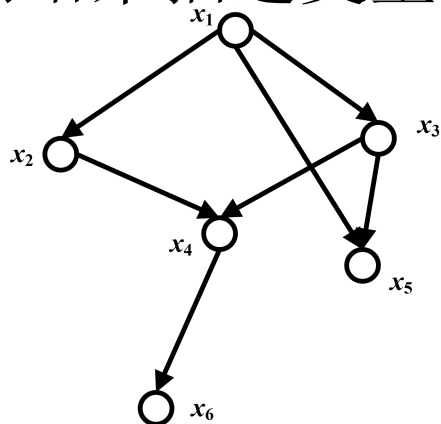
- ▣ COMIT采用一种树状结构来描述两量变量之间的关系,表达能力比MIMIC更强.



- ▣ 关键是如何确立变量之间树状关联关系的结构?
- ▣ 采用机器学习领域中Chow和Liu提出的方法构造概率模型

基于贝叶斯网络模型的EDA

- 采用贝叶斯网络来描述变量之间概率依赖关系



(其中节点代表变量，边表示变量之间的概率依赖关系)

- 基于贝叶斯网络的联合概率密度为:

$$P(x) = P(x_n | x_{n-1}, \dots, x_1) \cdot P(x_{n-1} | x_{n-2}, \dots, x_1) \cdot \dots \cdot P(x_2 | x_1) \cdot P(x_1)$$

基于贝叶斯网络模型的EDA

■ 关键问题

贝叶斯网络的结构如何确定？

■ 算法流程

//功能：BOA伪代码

procedure BOA

$D_0 \leftarrow$ Generate N individuals randomly

$l=1$

while not met stopping criterion **do**

$Se(l) \leftarrow$ select $se \leq N$ individuals from D_{l-1} according to a selection method

Construct the network B using a chosen metric and constraints

Generate a set of new individual $T(l)$ according to the joint distribution encoded by B

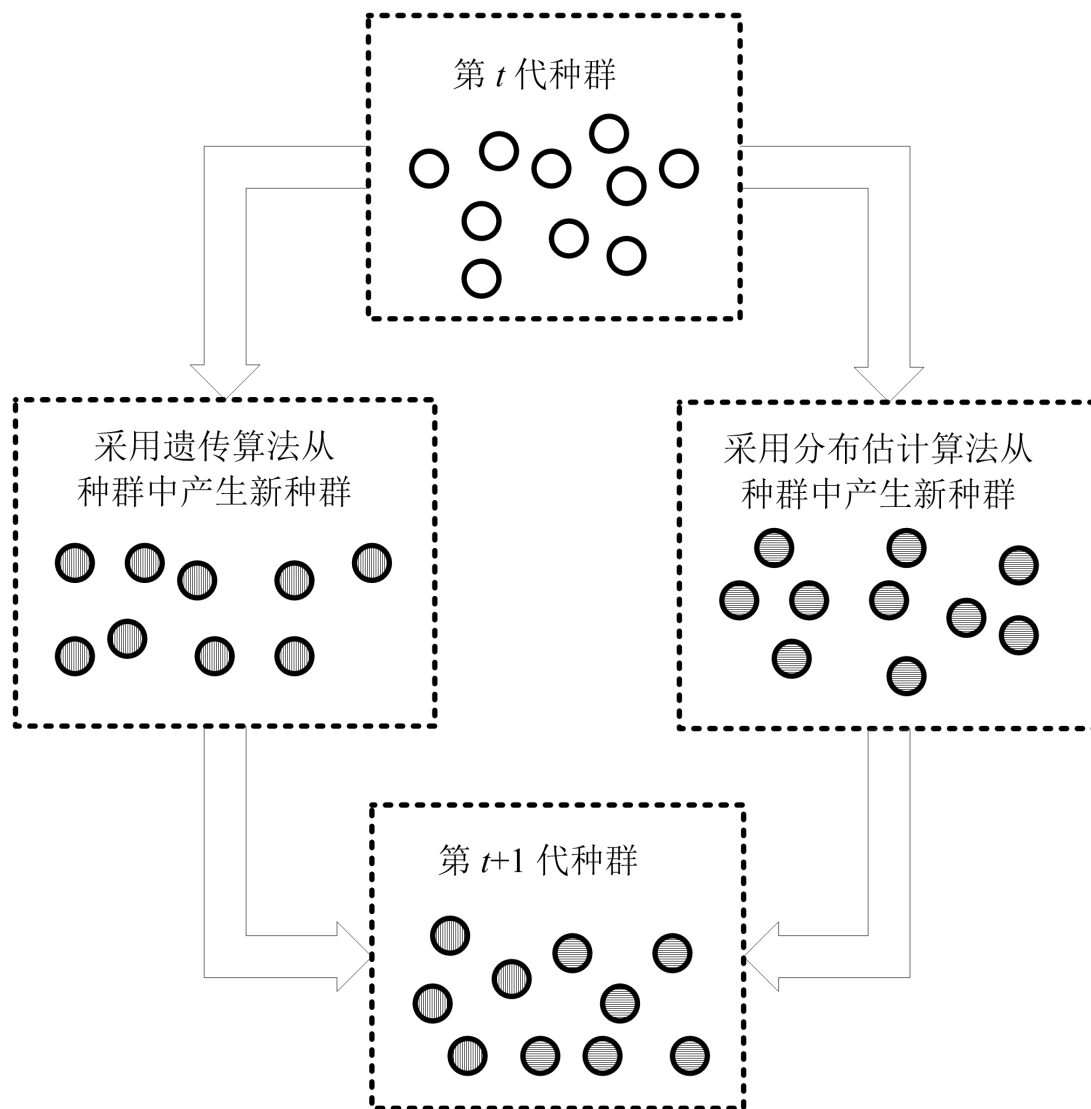
Create a new population D_l by replacing some individuals from D_{l-1} with $O(l)$

$l=l+1$

end while

end procedure

分布估计算法与遗传算法混合



分布估计算法与差分进化算法混合

第一步：从种群中选出 M 个较优的个体，建立如下概率模型

$$p_k(x) = \prod_{i=1}^n N(x_i; \hat{\mu}_i^k, \hat{\sigma}_i^k)$$

第二步：产生一个0~1之间的随机值 v ，若 $v \leq \alpha$ ，则按照DE方式产生新个体，否则按照EDA的方式取样产生新个体。

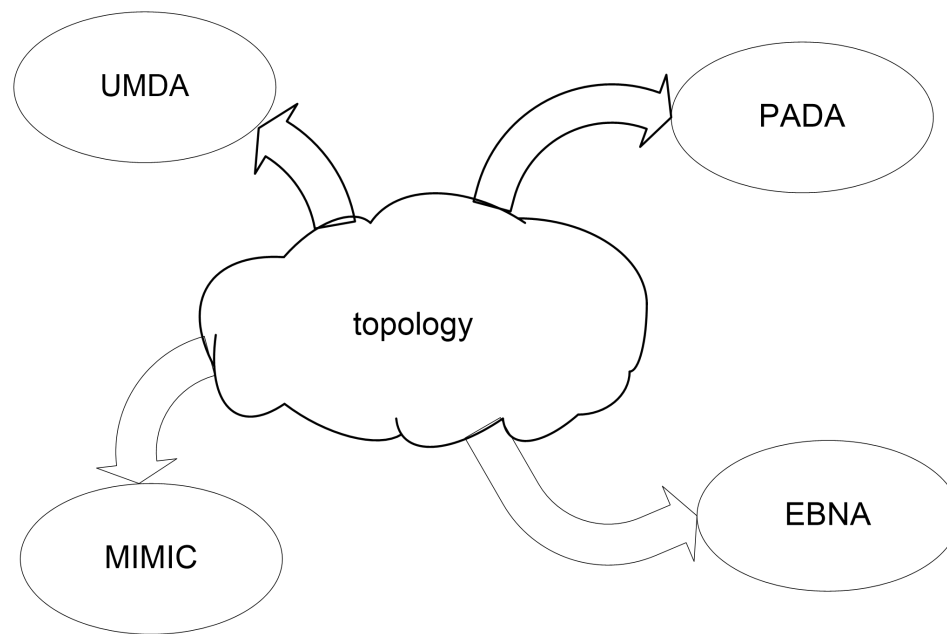
第三步：若新个体的适应度大于原个体的的适应度则替换之

第四步：若产生了足够数量的新种群则终止，否则执行第一步

并行分布估计算法(一)

■ 种群级别并行化

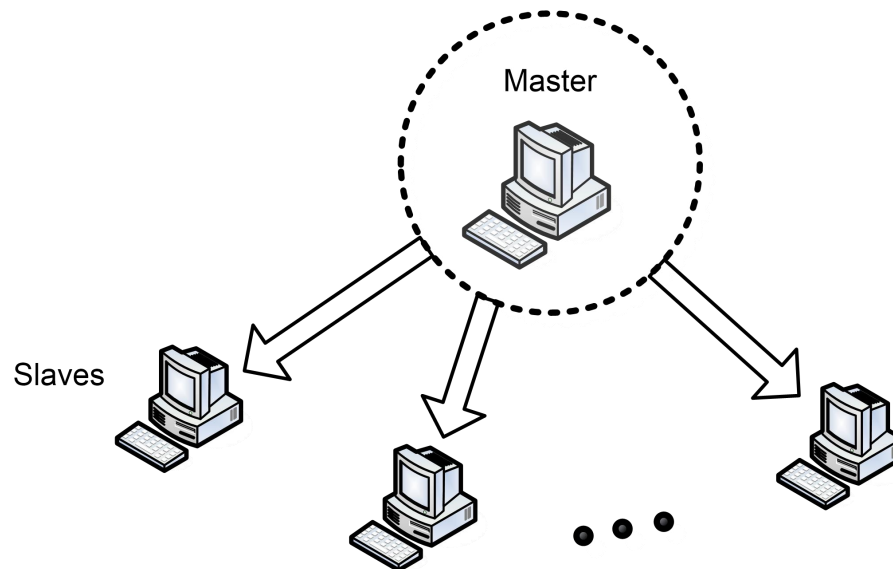
- 思路：将种群分成多个子种群，每个子种群在不同的机器上运行，然后各个子种群通过迁移等机制进行通信，达到综合信息的目的



并行分布估计算法(二)

■ 适应度评估并行化

- ▣ 思路:适应度评估通常是算法中最耗时的部分,因而,采用多台机器并行计算种群中的适应度可有效提高算法求解速度



主从模式并行计算

并行分布估计法(三)

■ 概率模型构建并行化

- 思路:设计复杂的概率模型需要较大的计算量, 并行求解复杂概率模型可有效提高算法计算速度.

■ 其他并行机制

- 采样的操作进行并行化
- 混合并行机制

分布估计算法的理论研究

研究者	说明
Markus Hohfeld(1997)[63]	证明PBIL在解决线性的二进制优化问题时，可以收敛到全局最优解，解非线性问题可能会陷入局部最优。
H. Muhlenbein(1998)[64]	对UMDA的收敛性进行分析，假设种群无穷大。
H. Muhlenbein(1998)[65]	对FDA的收敛性进行分析。
M.Pelikan(2001)[66]	对BOA解决OneMax问题的收敛性进行分析。
Q.F. Zhang(2004)[67]	对种群规模无穷大的EDA进行数学建模，分析了EDA算法达到全局收敛的一些条件。
R.Rastegar(2005)[68]	分析了种群规模无穷大的EDA要达到全局收敛所需要的代数。
Tianshi Chen(2007)[69]	对早期的两个分布估计算法：UMDA和增量UMDA进行时间复杂度的分析。
Jiri Ocenasek(2006)[70]	提出用熵来度量分布估计算法收敛性的方法，并在此基础上分析了EDA终止的条件。
...	...

分布估计算法的应用领域(函数优化)

- 有效地保护好模式，能够高效求解高维的复杂函数优化问题.
- 在具有先验知识的情况下，可有针对地选择概率模型，从而设计出性能优越的分布估计算法.
- 已成功应用于求解复杂的多峰函数，关联性强的复杂函数优化以及多目标函数优化.

分布估计算法的应用领域(组合优化)

应用（中文）	英文	参考文献
旅行商问题	Traveling Salesman Problem	Larrañaga (2002)[33]
作业调度问题	Job Shop Scheduling Problem	Larrañaga (2002)[33] 、 Jarboui(2008)[38]
护士调度问题	Nurse Scheduling Problem	U. Aickelin(2006)[39]
网络控制	Network Control	H.B. Li(2008)[37]
最大团问题	Maximum Clique Problem	Q.F. Zhang(2005)[34]
核反应堆燃料管理问题	Nuclear Reactor Fuel Management	S.Jiang(2006)[36]
最大分集问题	Maximum Diversity Problem	Wang(2009)[35]

分布估计算法的应用领域(生物信息学及其它)

■分布估计算法 在处理高维复杂问题时表现出良好的性能，非常适合于求解含有海量信息的生物信息学领域的问题。

- ▣基因结构分析

- ▣DNA微阵列数据的分类和聚类分析

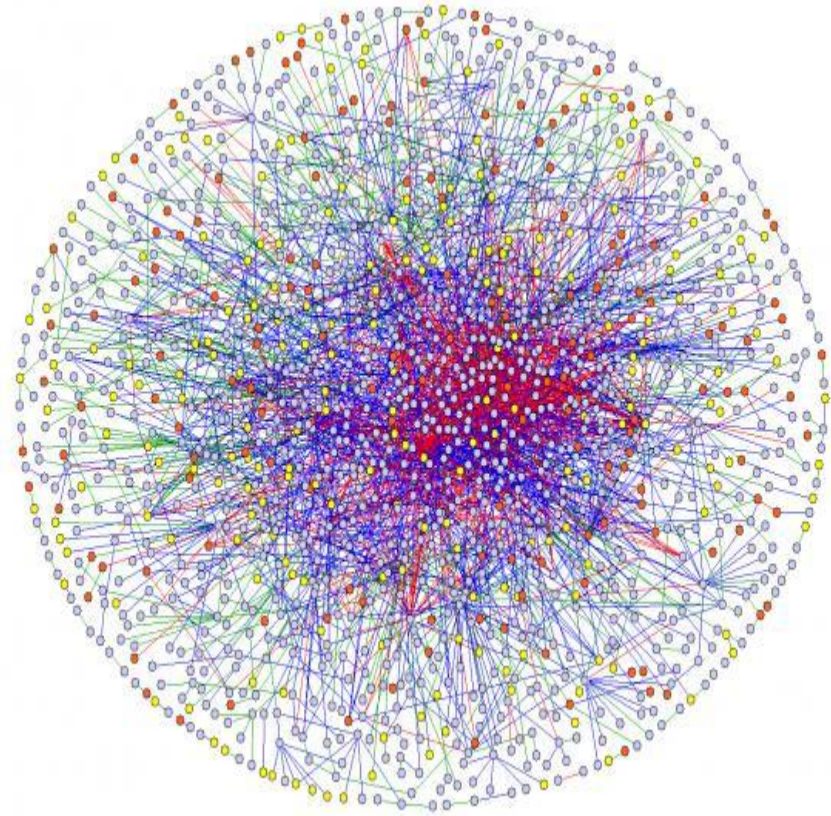
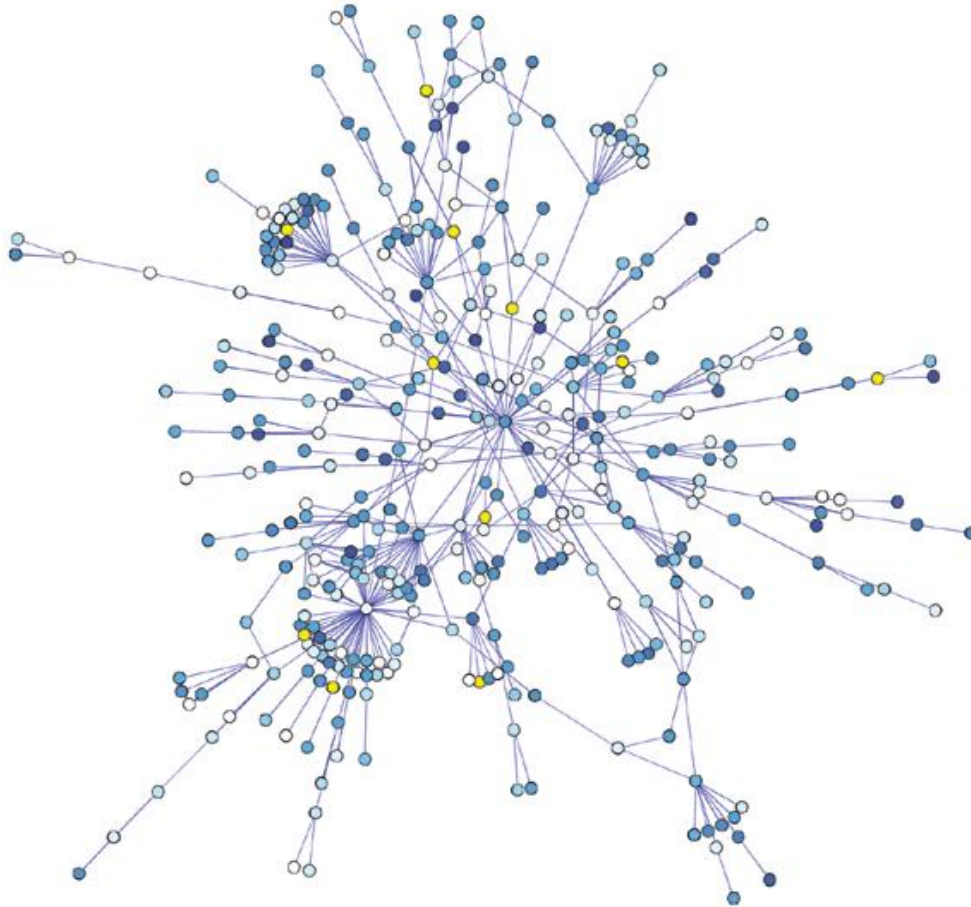
- ▣蛋白质结构预测与蛋白质设计

■分布估计算法在多目标优化、机器学习、模式识别和聚类分析等领域也取得了成功的应用。

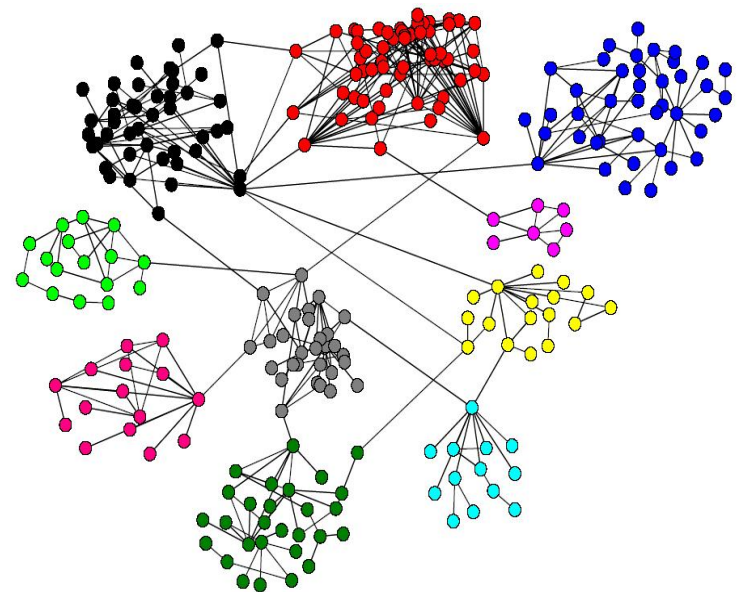
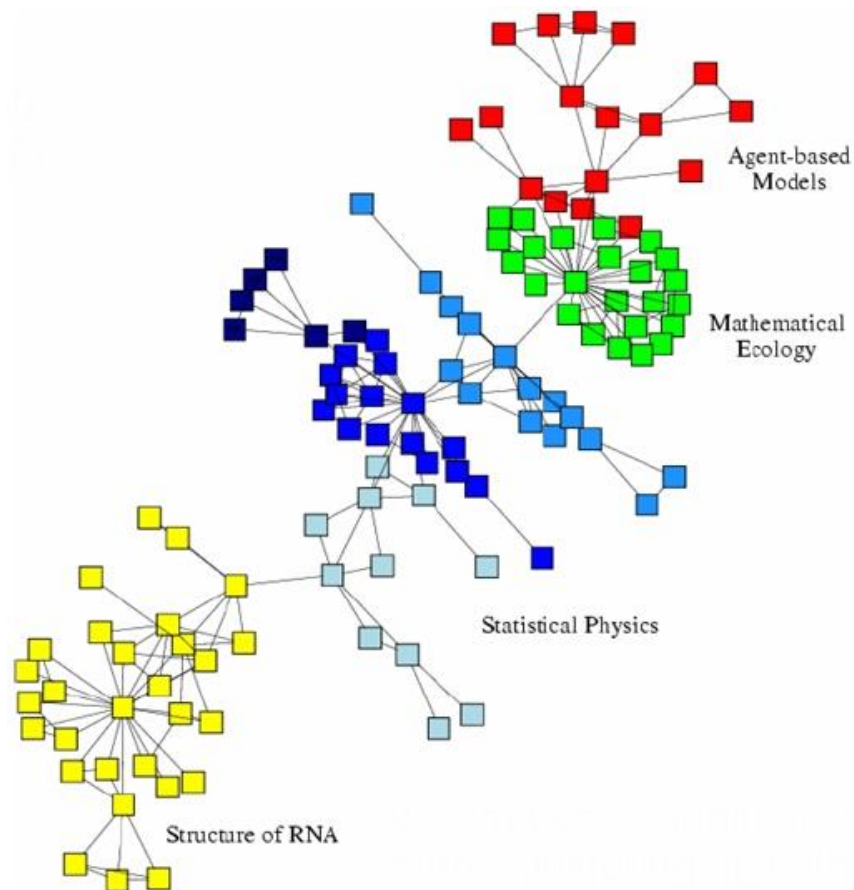
第六节：复杂网络



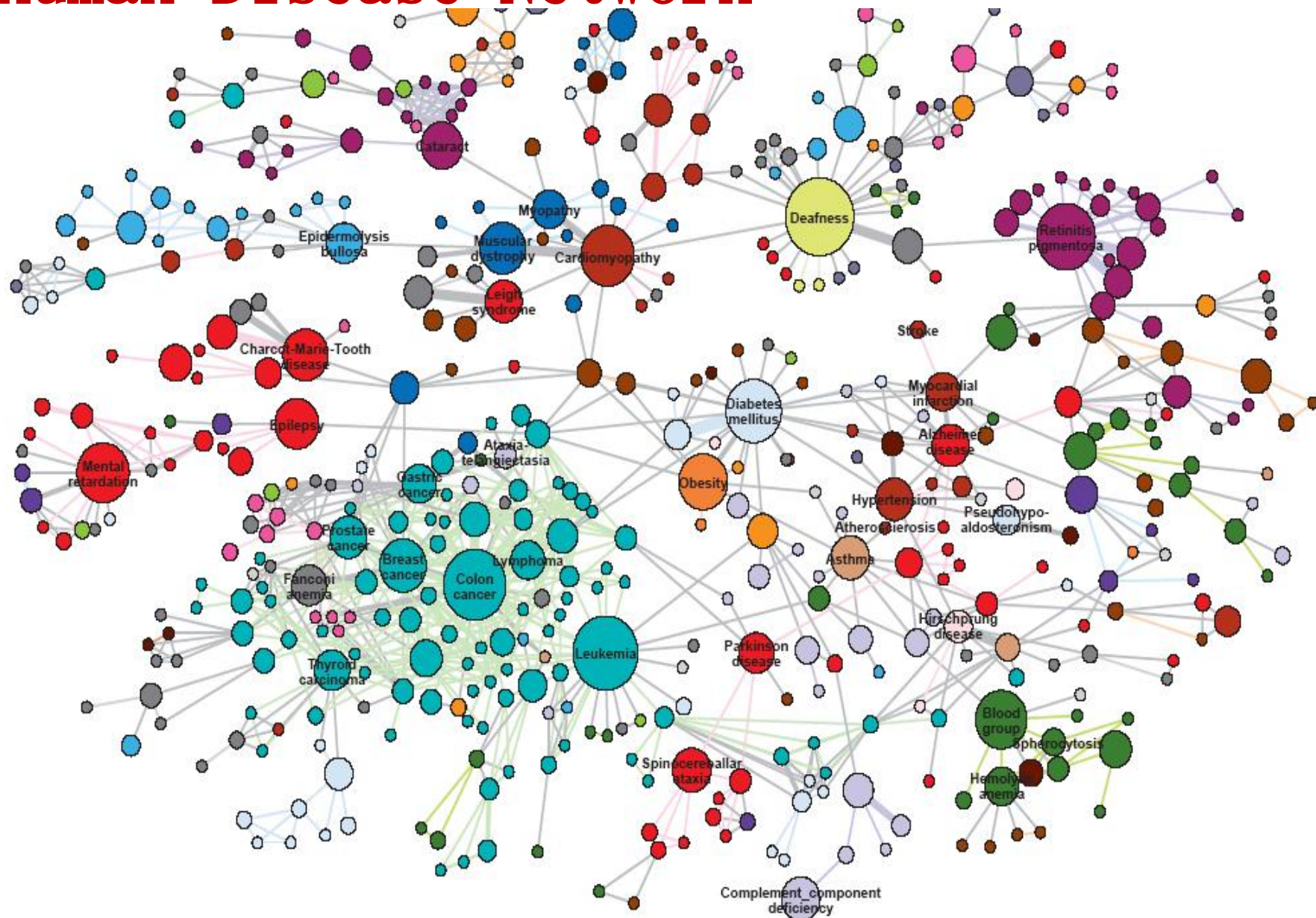
Protein-protein interaction networks

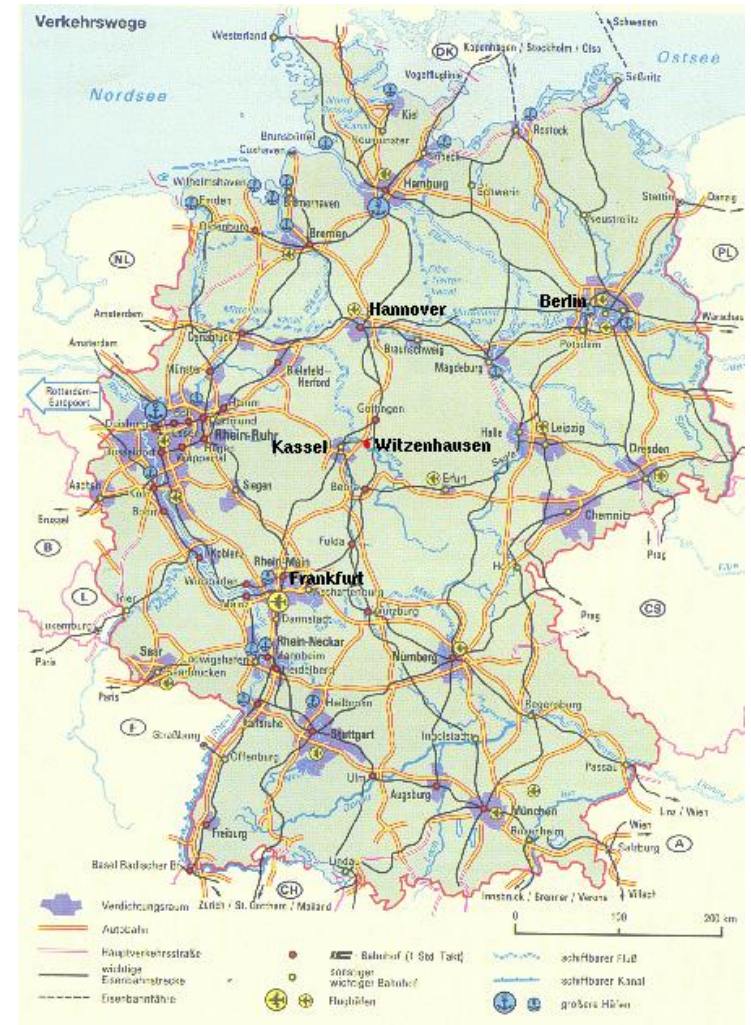
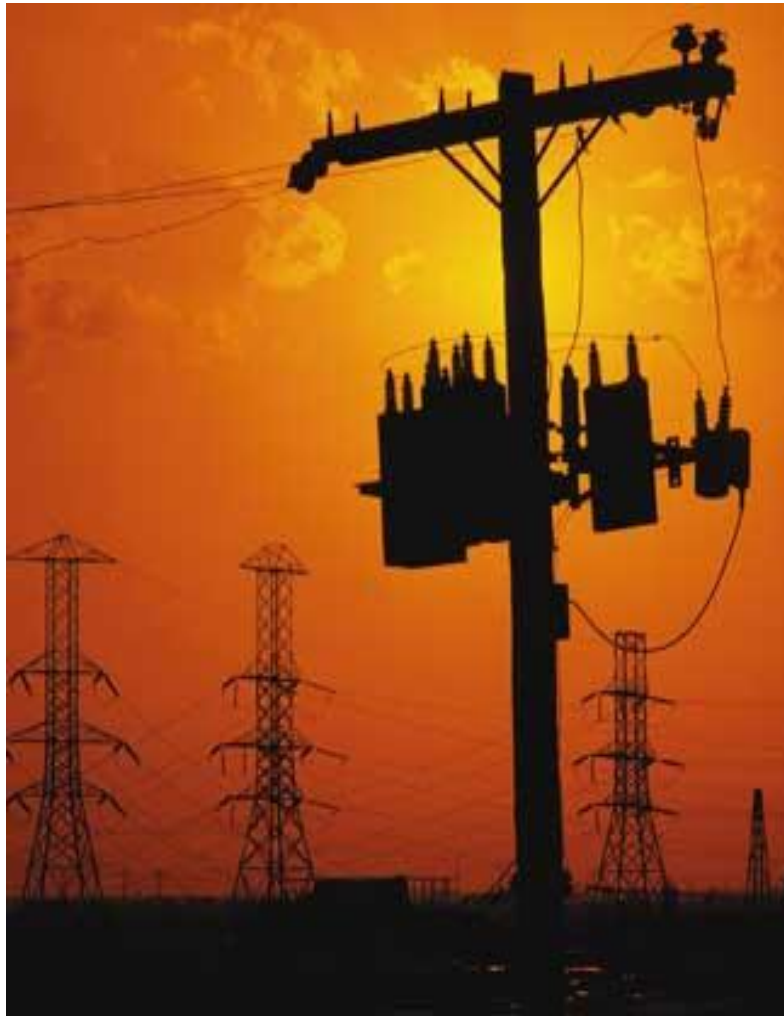


Scientist collaboration networks



Human Disease Network





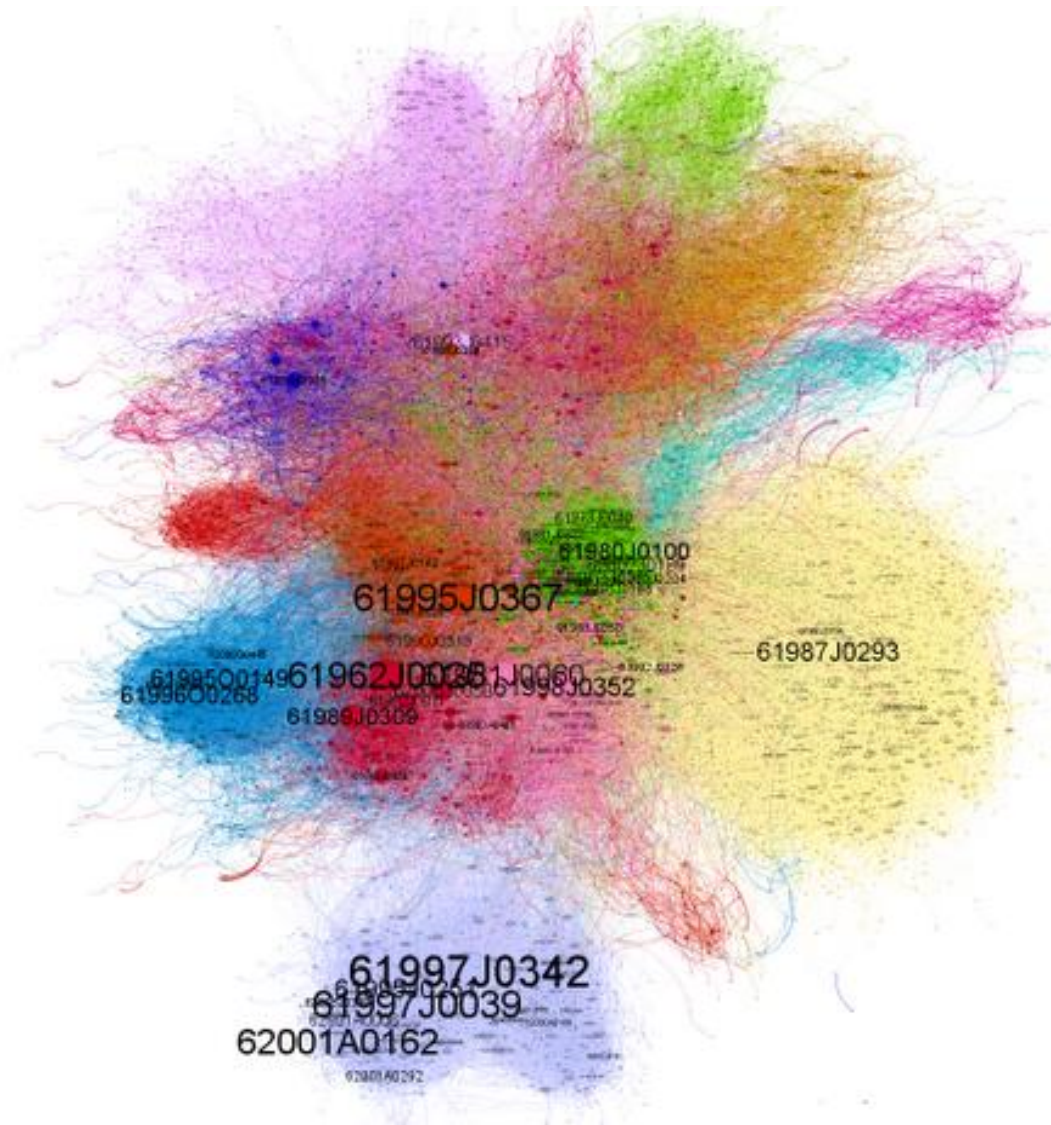
Network

⌘ Behind a complex system there is a **network**, that defines the interactions between the component.

⌘ A **network** can be represented as a graph.

⌘ “大数据面临的科学问题本质上可能就是网络科学问题，复杂网络分析应该是数据科学的重要基石”

————— 李国杰 院士 《大数据研究的科学价值》



□ Degree Distributions

Scale-free/Small-world

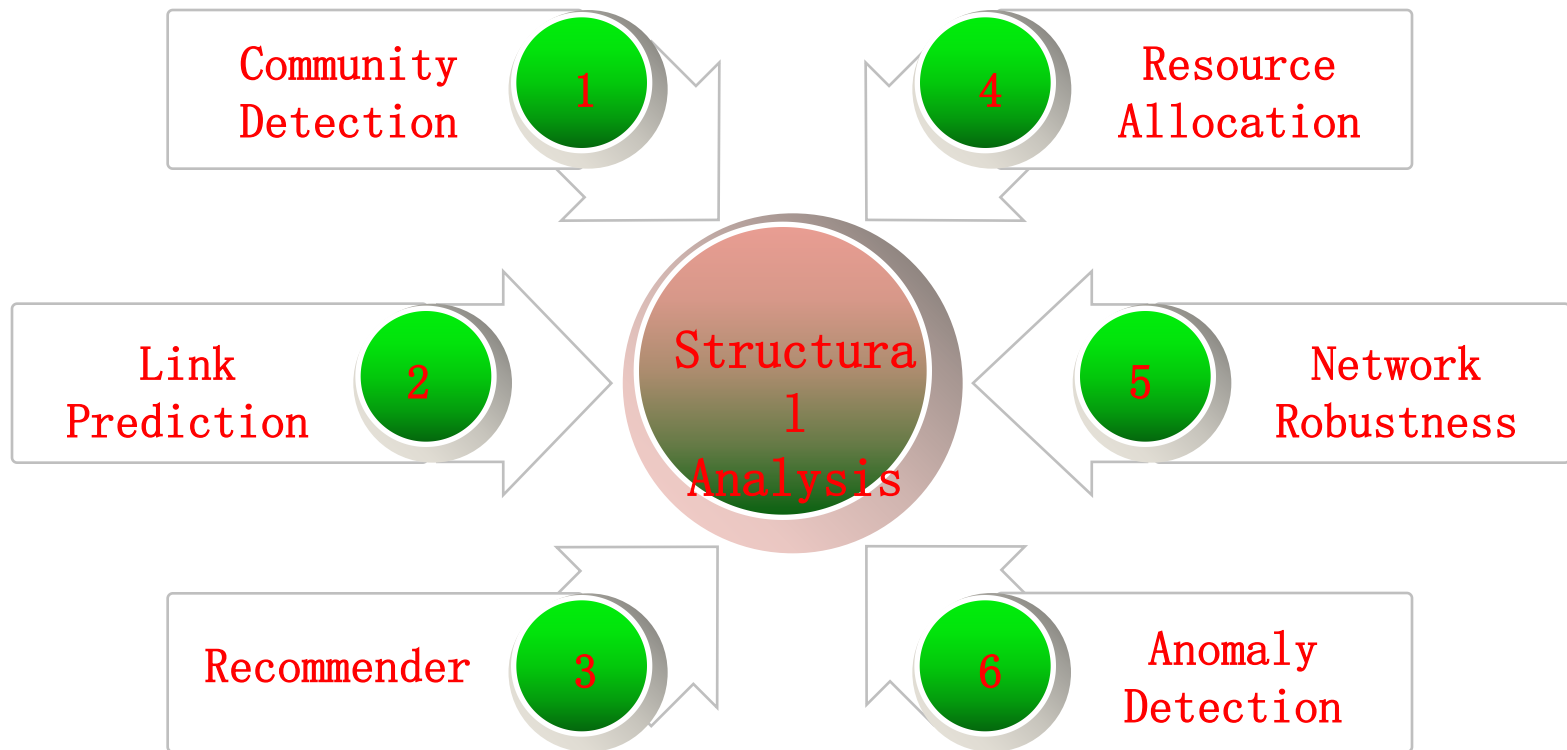
□ Dynamics

□ Community Structure

□ Recommender

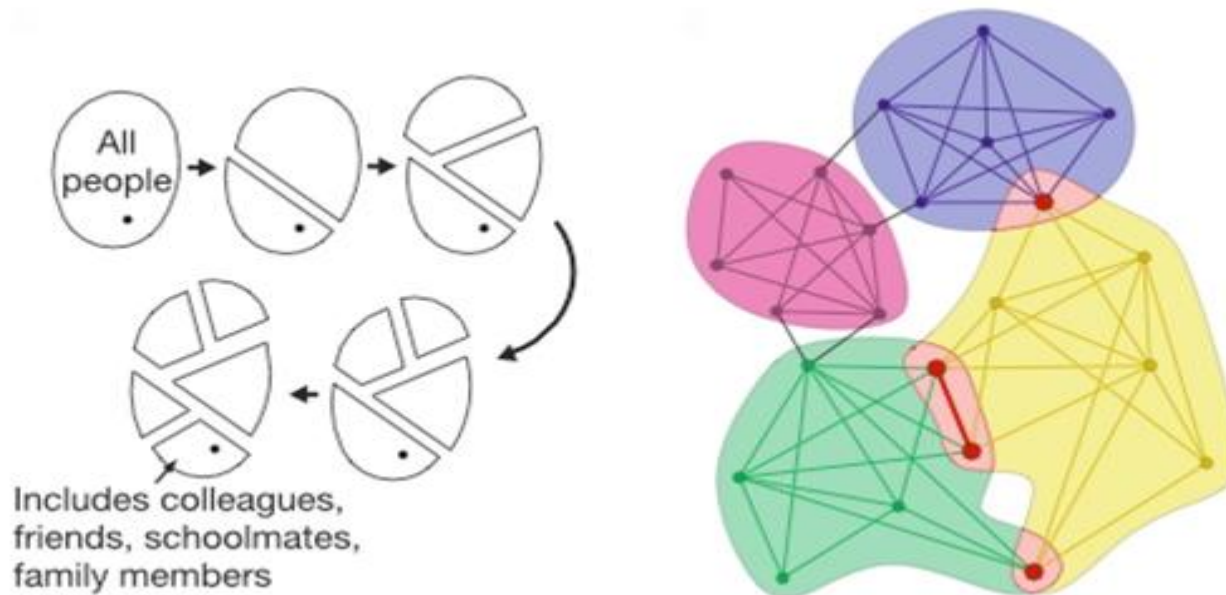
□

Network Problems



Community structure

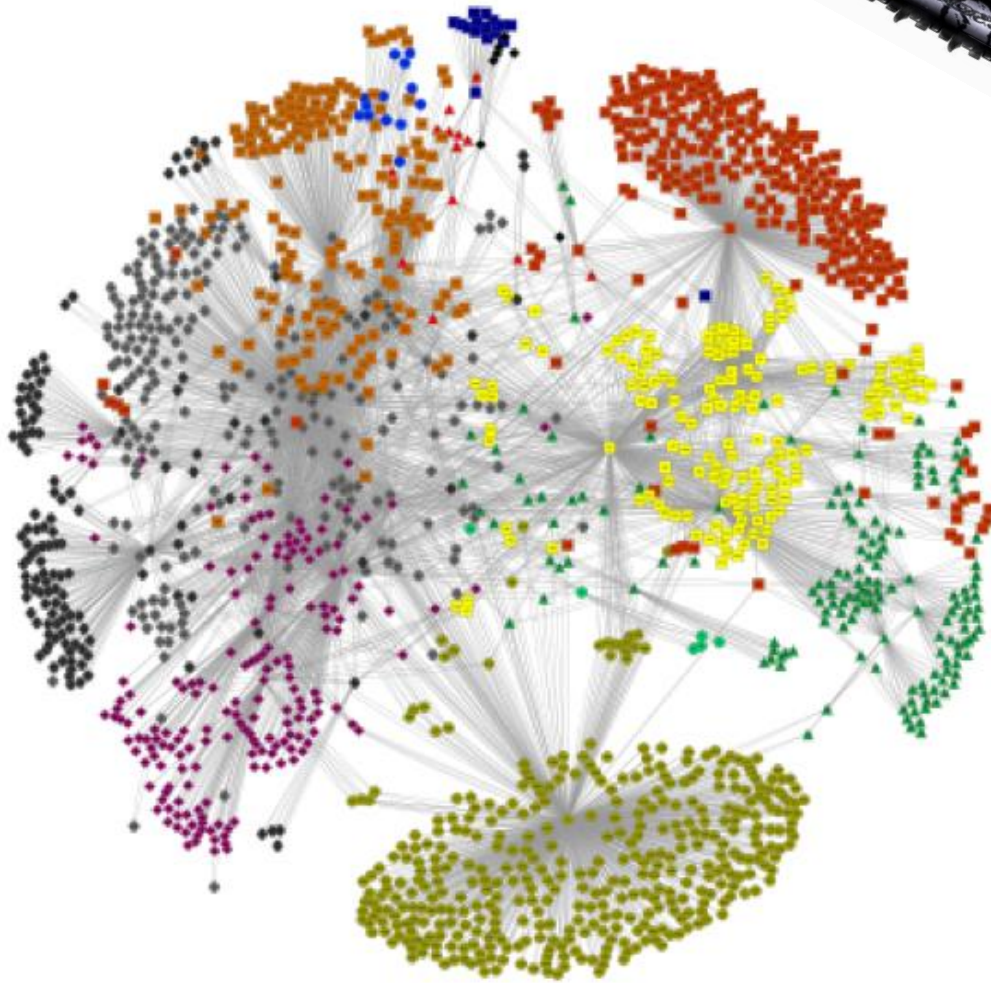
Community structure is the groups of network vertices. Within these groups there have **dense internal links**, but **between groups** there are **fewer edges**.

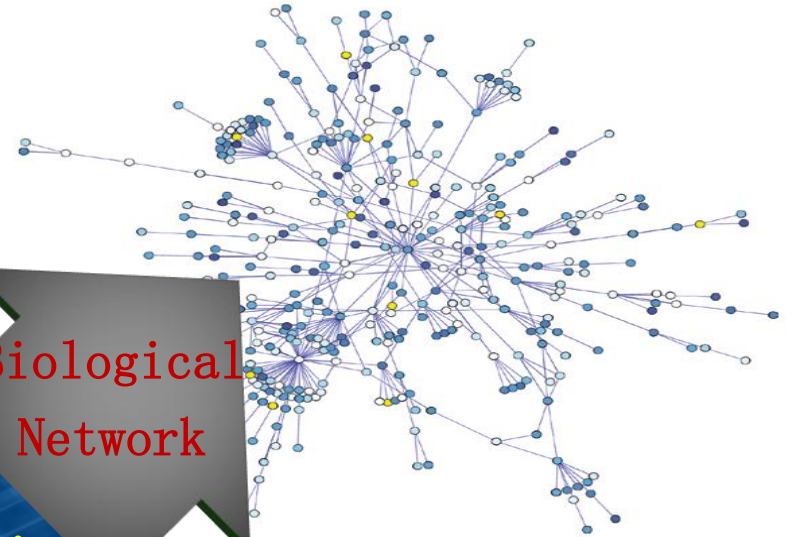
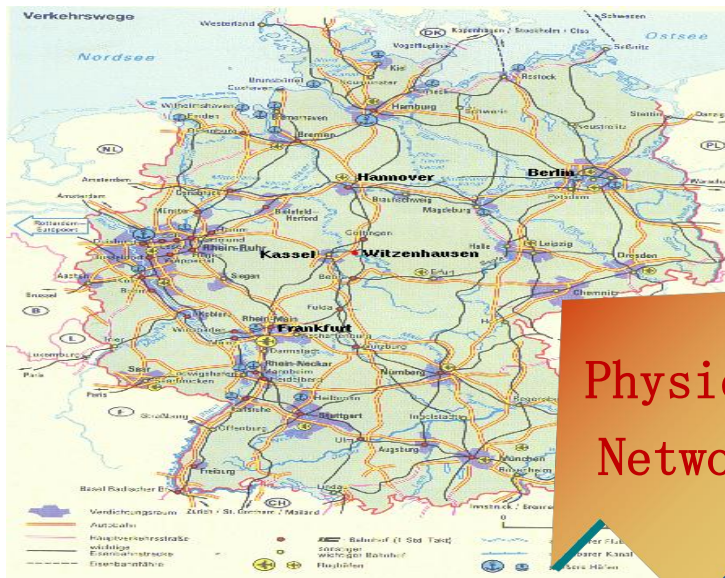


Girvan, Newman. *PNAS* (2002)
Palla et al. *Nature* (2005)



Community Detection





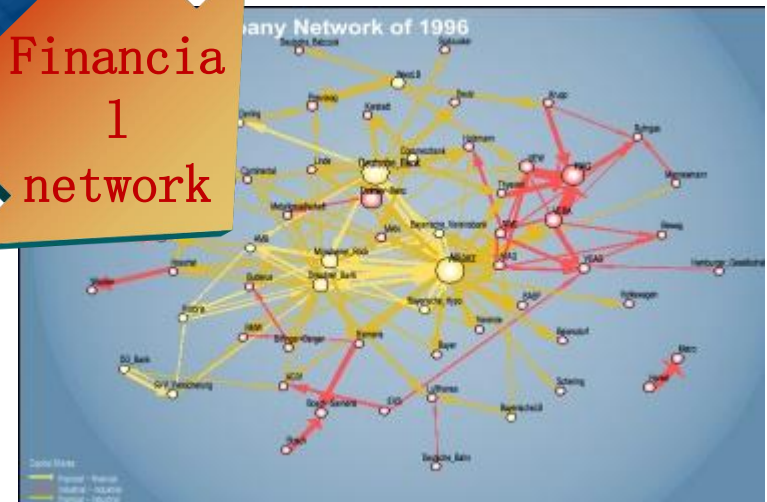
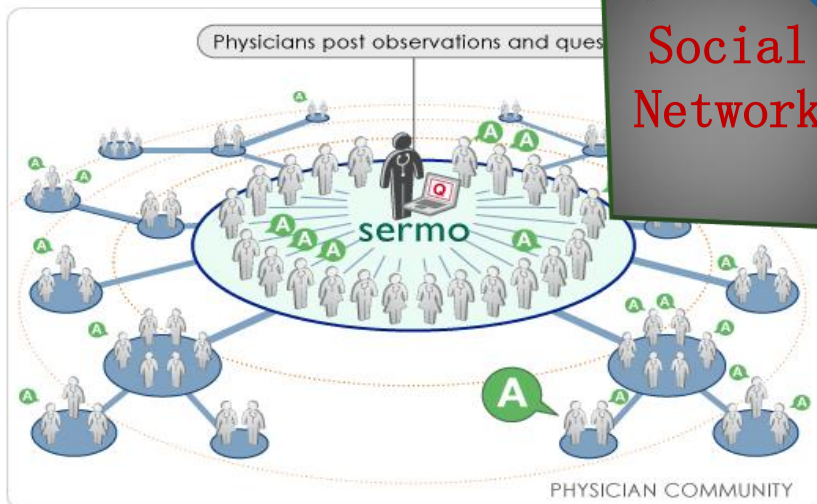
Physical Network

Biological Network

Community Detection

Social Network

Financia 1 network



Community Detection Methods

⌘ Graph-based Methods

- ⌘ GN algorithm: Girvan, Newman, *PNAS* (2002)
- ⌘ Minimum-cut: Newman, *Eur. Phys. J. B* (2004)
- ⌘ Spectral analysis: Donetti, *Stat. Mech.* (2004)

⌘ Dynamics-based Methods

- ⌘ Synchronization: Arenas, *PRL* (2006)
- ⌘ Random Walk: Rosvall, *PNAS* (2007, 2008); Lai, *PRE* (2011)
- ⌘ Clustering dynamics: Wu, *PRE* (2012)

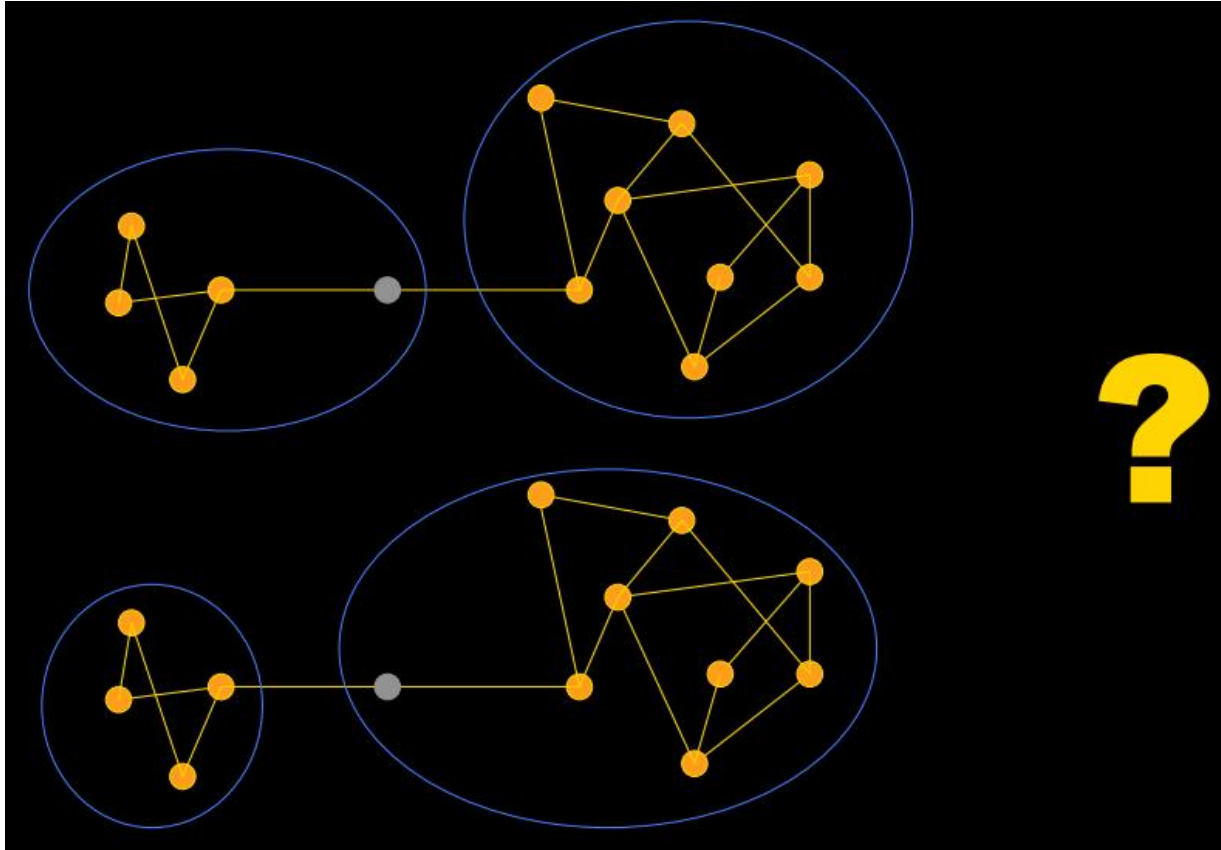
⌘ Optimization-based Methods

- ⌘ Greedy algorithm: Newman, *PRE* (2004); Clauset, *PRE* (2004); Schuetz, *PRE* (2008)
- ⌘ Simulated annealing: Amaral, *PRE* (2004); Massen, *PRE* (2005)
- ⌘ Evolutionary algorithm: ? Pizzuti, *GECCO* (2008)

⌘



How to evaluate a partition?



Modularity (模件性)

Modularity is the fraction of the edges that fall within the given groups minus the expected such fraction if edges were distributed at random.

$$Q = \frac{1}{L} \sum_{s=1}^m \left(l_s - \frac{d_s^2}{4L} \right)$$

l_s : # links inside community s

L : # links in the network

d_s : The total degree of the nodes in community s

$\frac{d_s^2}{4L}$: Expected number of links in
community s

Newman and Girvan, *PRE* (2004)



Modularity Maximization

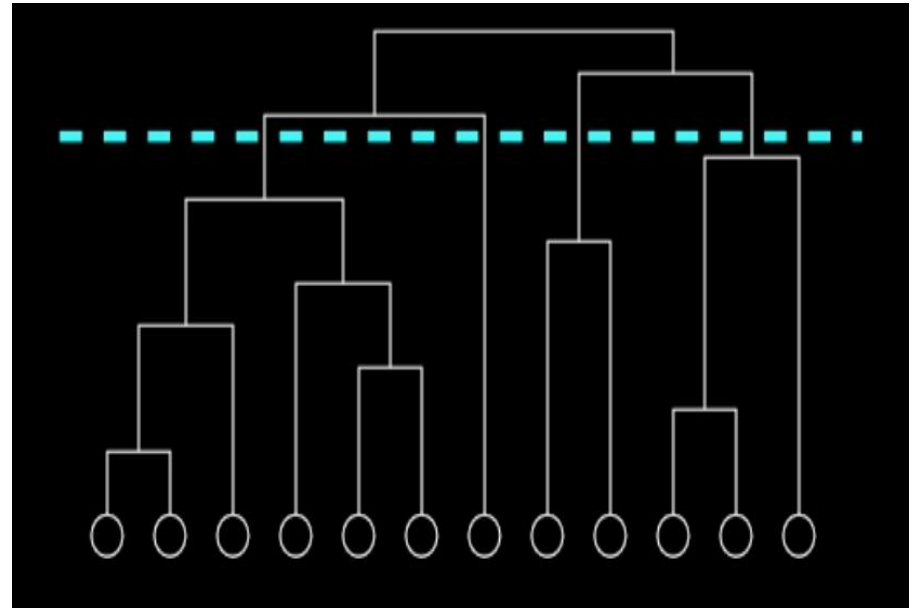
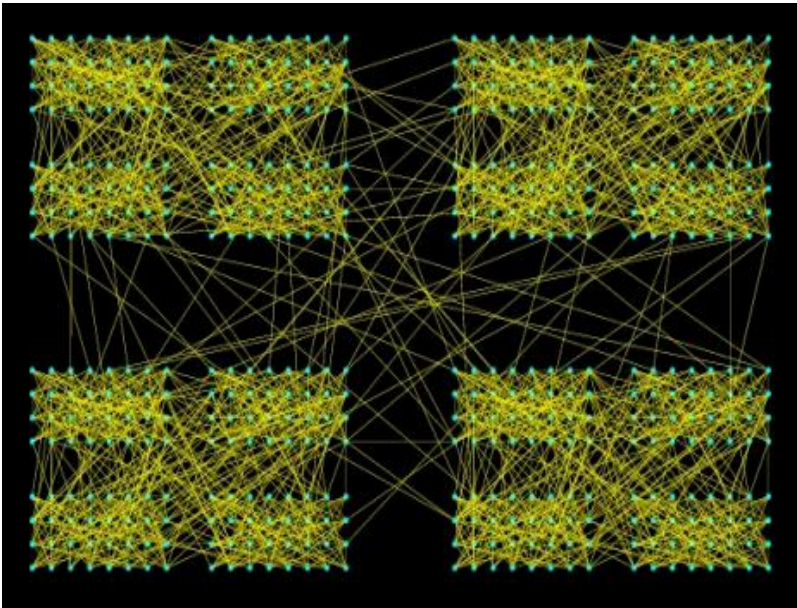
$$\text{Max } Q = \frac{1}{L} \sum_{s=1}^m \left(l_s - \frac{d_s^2}{4L} \right)$$

NP-complete!

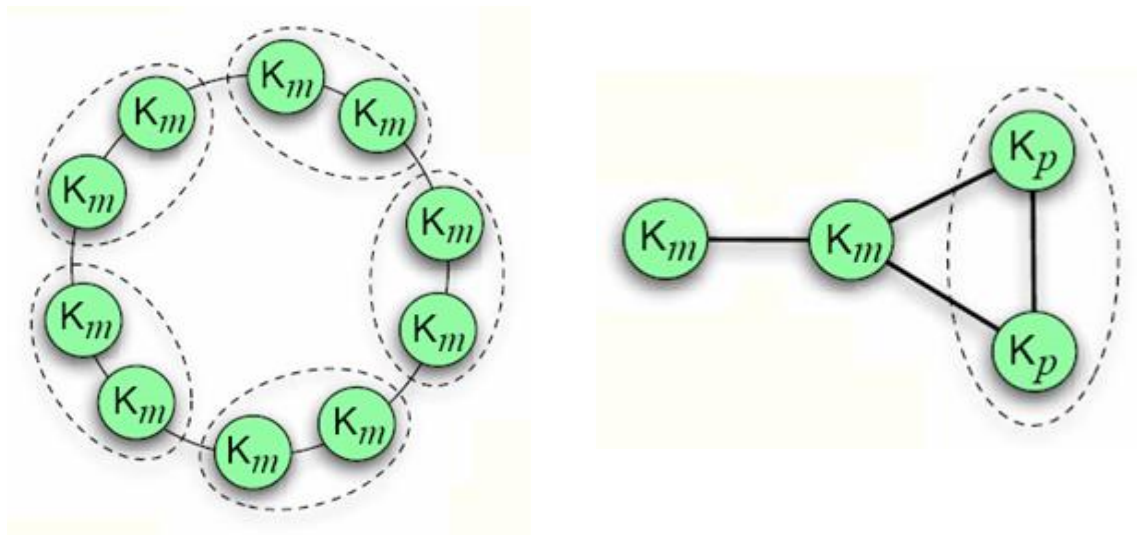
Newman, Phys. Rev. E 69, 066133, 2004

Multi-resolution problem in Modularity

- ✂ Only one solution.
- ✂ Partition corresponding to the maximum modularity may be not the one we expected.



Resolution limit in Modularity



⌘ Modularity optimization may **fail to identify modules** smaller than a scale which depends on the total size of the network and on the degree of interconnectedness of the modules

⌘ Fortunato & Barthélemy, *PNAS* (2007)

A Memetic Algorithm for Community Detection

⌘ Motivation

- ⌘ Optimizing a metric instead of Q to overcome the resolution limit and to discover community structure at multiple resolution levels
- ⌘ Combining GA and problem-specific knowledge

M. Gong, B. Fu, L. Jiao, H. Du. Memetic algorithm for community detection in networks. *Phys. Rev. E*, 2011.

density

⌘ Modularity density: the sum over the clusters of the ratio between the difference of the internal and external degrees of the cluster and the cluster size.

$$D = \sum_{i=1}^m \frac{L(V_i, V_i) - L(V_i, \bar{V}_i)}{|V_i|}$$

⌘ General modularity density measure

$$D_\lambda = \sum_{i=1}^m \frac{\lambda L(V_i, V_i) - (1 - \lambda) L(V_i, \bar{V}_i)}{|V_i|}$$

Z. Li, S. Zhang, et al. Quantitative function for community detection.
Phys. Rev. E, 2008

Algorithm framework

Algorithm 1 Algorithm framework of Meme-Net

- 1: **Input:** Maximum number of generations: $G_{\max}$; Population size: S_{pop} ; Size of mating pool: S_{pool} ; Tournament size: S_{tour} ; Crossover probability: P_c ; Mutation probability: P_m .
 - 2: $\mathbf{P} \leftarrow \text{GenerateInitialPopulation}(S_{\text{pop}})$;
 - 3: **repeat**
 - 4: $\mathbf{P}_{\text{parent}} \leftarrow \text{Selection}(\mathbf{P}, S_{\text{pool}}, S_{\text{tour}})$;
 - 5: $\mathbf{P}_{\text{child}} \leftarrow \text{GeneticOperation}(\mathbf{P}_{\text{parent}}, P_c, P_m)$;
 - 6: $\mathbf{P}_{\text{new}} \leftarrow \text{LocalSearch}(\mathbf{P}_{\text{child}})$;
 - 7: $\mathbf{P} \leftarrow \text{UpdatePopulation}(\mathbf{P}, \mathbf{P}_{\text{new}})$;
 - 8: **until** $\text{TerminationCriterion}(G_{\max})$
 - 9: **Output:** Convert the fittest chromosome in $\mathbf{P}$ into a partition solution and output.
-

Representation and Initialization

✧ Representation:

$$\mathbf{x} = \{x^1 \ x^2 \ \dots \ x^n\}$$

✧ Population initialization

For each chromosome, we randomly select a vertex and assign its cluster identifier to all of its neighbors.

Crossover and Mutation

⌘ Crossover

$$x_b^k \leftarrow x_a^i, \forall k \in \{k \mid x_a^k = x_a^i\}$$

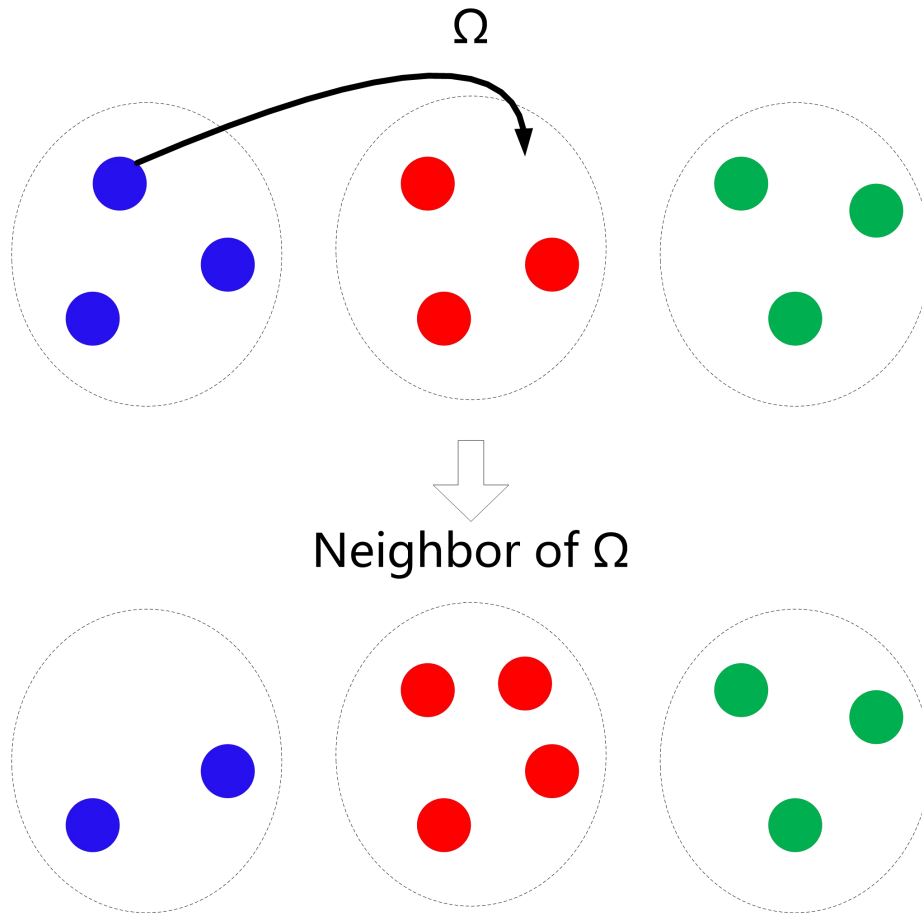
$$x_a^k \leftarrow x_b^i, \forall k \in \{k \mid x_b^k = x_b^i\}$$

v	$\mathbf{x}_a$		$\mathbf{x}_b$		$\mathbf{x}_c$		$\mathbf{x}_d$	$\mathbf{x}_a$		$\mathbf{x}_b$		v	
1	⑤	→	2	→	⑤	5		5		2		1	
2	3		6		6	⑥	←	3	←	⑥		2	
③	→	⑤	→	6	→	⑤	⑥	←	5	←	⑥	←	③
4	7		5		5	7		7		5		4	
5	2		6		6	⑥	←	2	←	⑥		5	
6	⑤	→	3	→	⑤	5		5		3		6	
7	3		2		2	3		3		2		7	

⌘ Neighbor-based mutation

Local search

Definition of neighbors of a partition.



$$N_{\Omega} = n(m - 1)$$

Local search

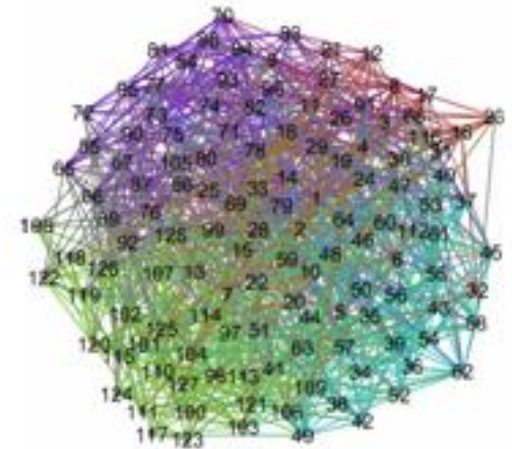
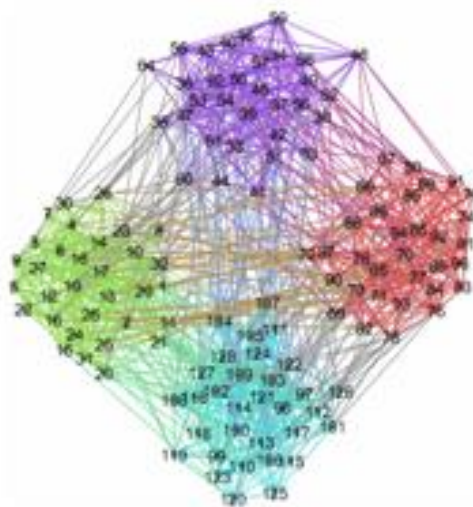
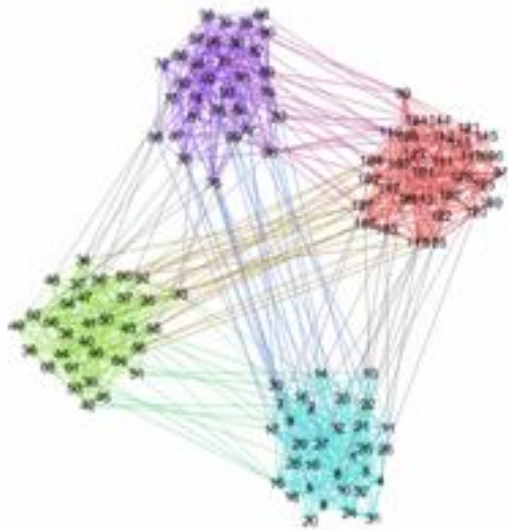
- ❑ When should local search be applied within the evolutionary cycle?
- ❑ Which individuals in the population should be improved by local search?
- ❑ How much computational effort should be allocated to each local search?

Local search

Algorithm 3 Local search procedure

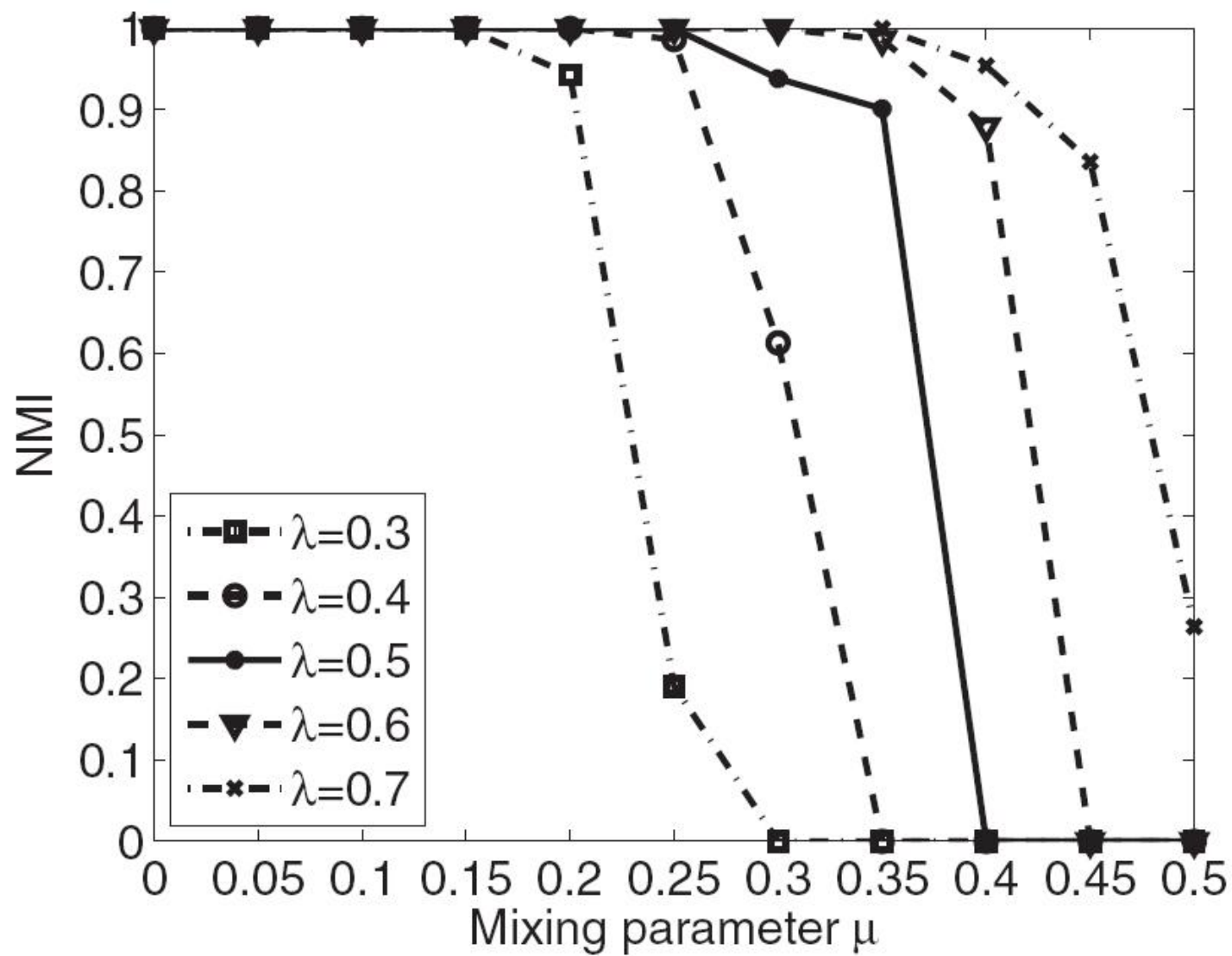
```
1: Input:  $\mathbf{P}_{child}$ .
2:  $\mathbf{n}_{current} \leftarrow \text{FindBest}(\mathbf{P}_{child})$ ;
3:  $y_{islocal} \leftarrow \text{FALSE}$ ;
4: repeat
5:    $\mathbf{L} \leftarrow \text{FindNeighbors}(\mathbf{n}_{current})$ ;
6:    $\mathbf{n}_{next} \leftarrow \text{FindBest}(\mathbf{L})$ ;
7:   if  $\text{Eval}(\mathbf{n}_{next}) > \text{Eval}(\mathbf{n}_{current})$  then
8:      $\mathbf{n}_{current} \leftarrow \mathbf{n}_{next}$  ;
9:   else
10:     $y_{islocal} \leftarrow \text{TRUE}$ ;
11:  end if
12: until  $y_{islocal}$  is TRUE
13: Output:  $\mathbf{P}_{child}$  .
```

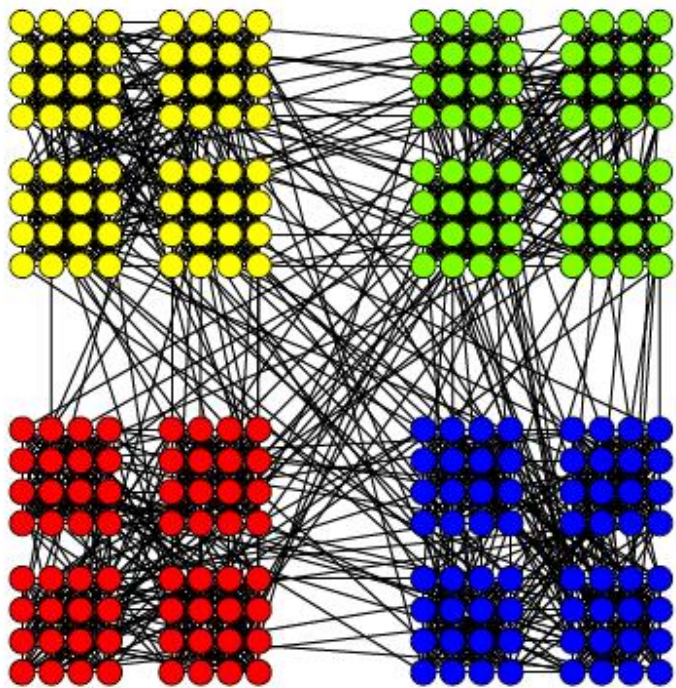
Benchmark Networks



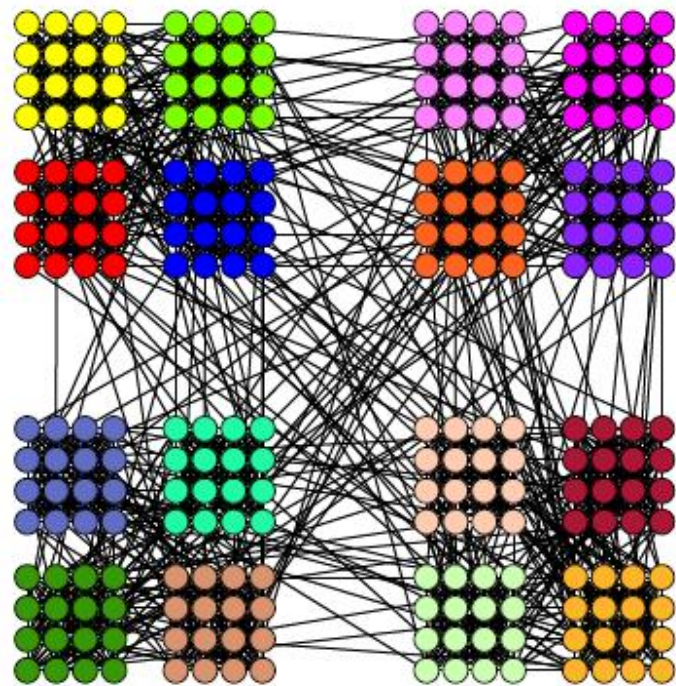
μ

Lancichinetti, *PRE* (2008)

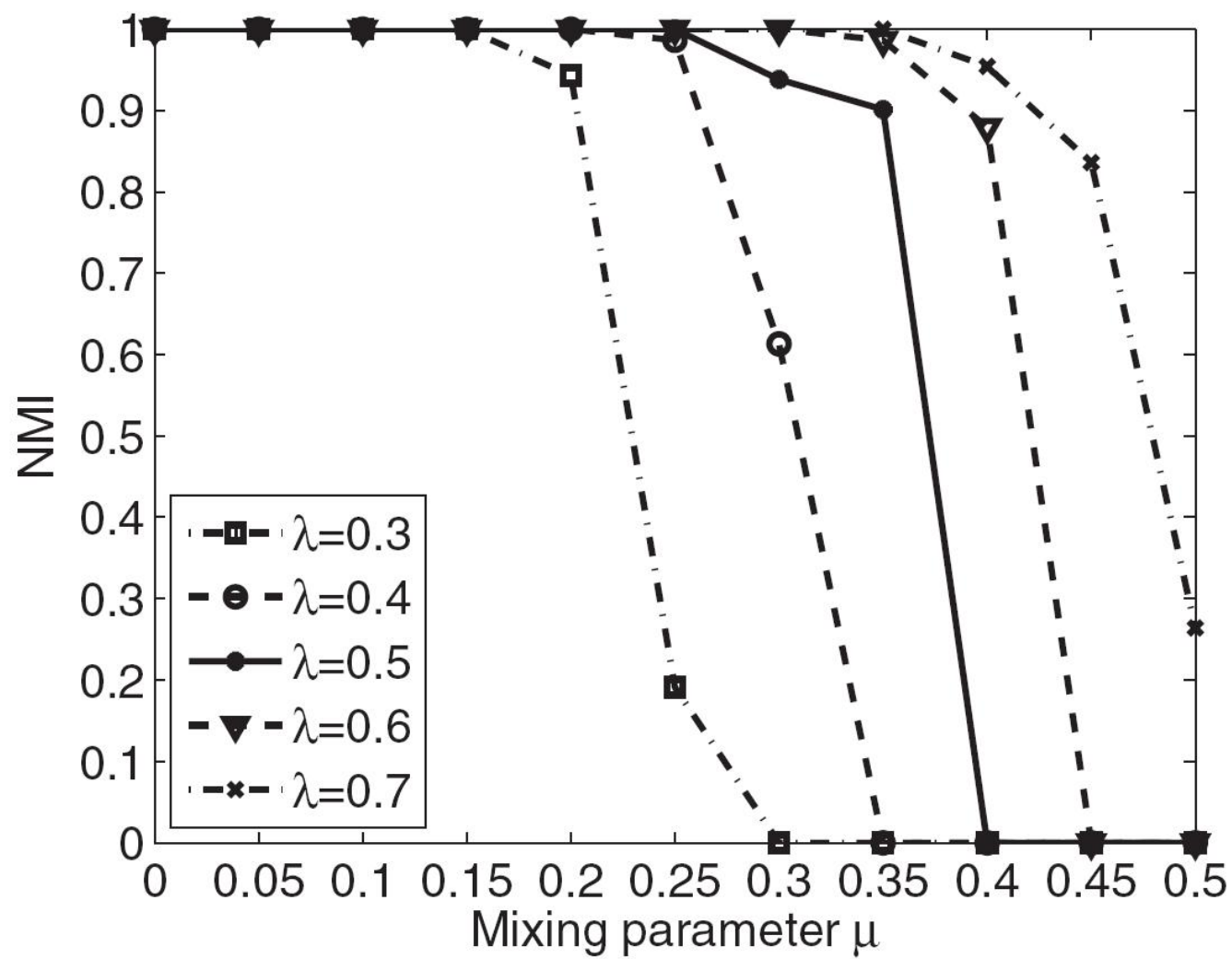




$\lambda = 0.3$

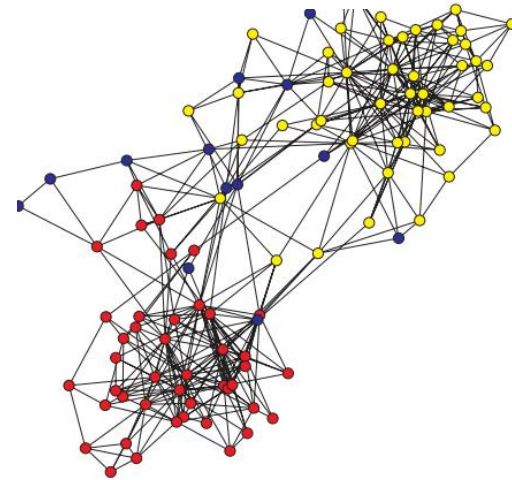
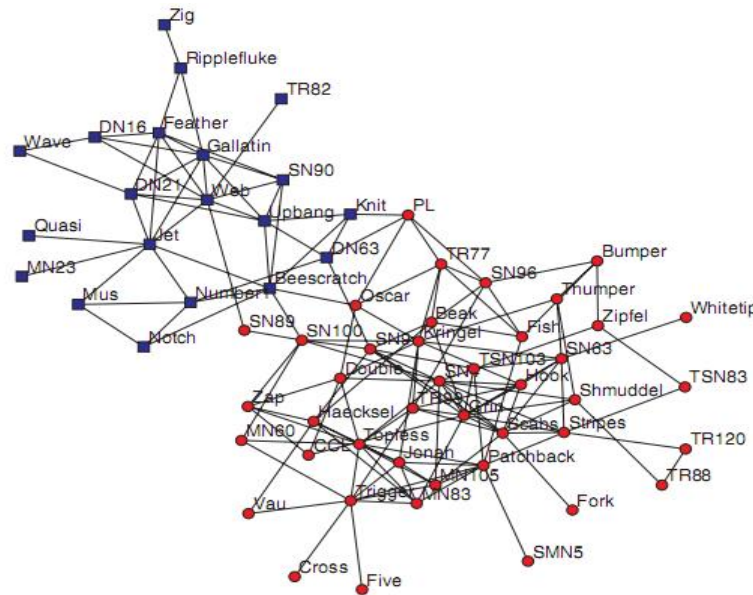
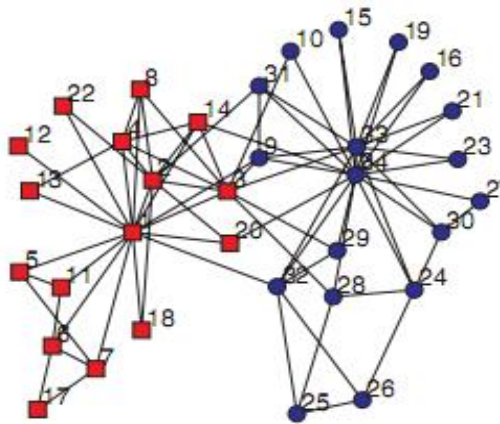
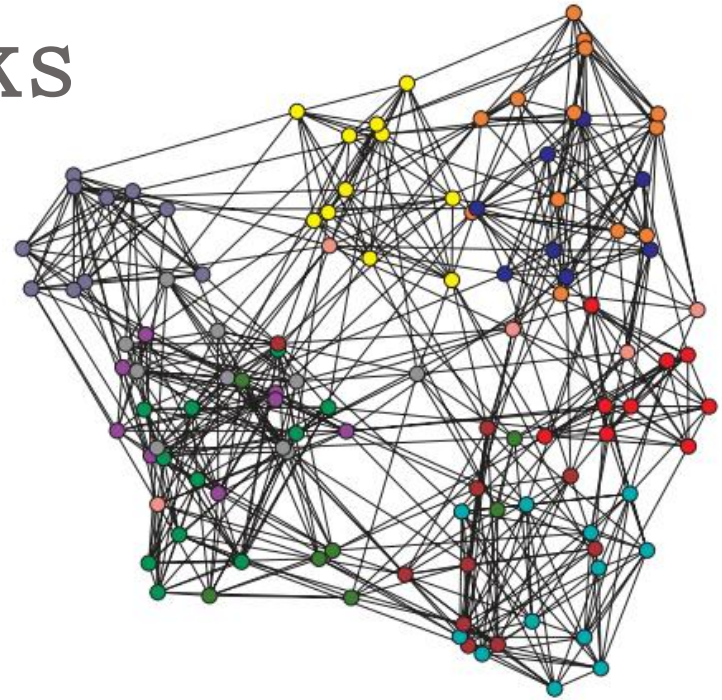


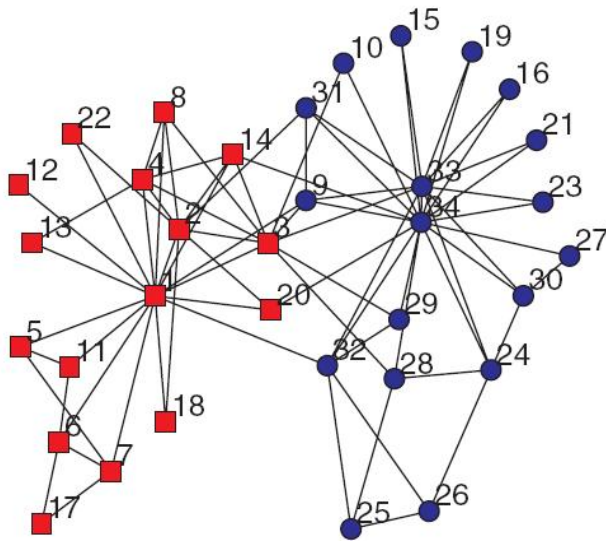
$\lambda = 0.7$



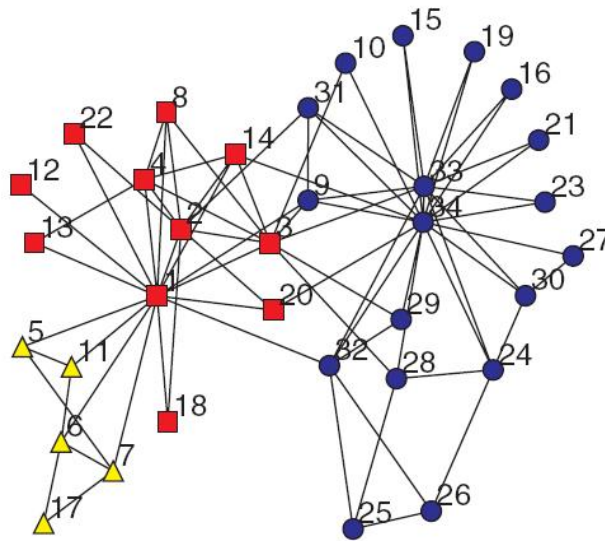
Real-world Networks

- ⌘ Zachary's karate club
- ⌘ Dolphin social network
- ⌘ American college football
- ⌘ Books about US politics





(a)



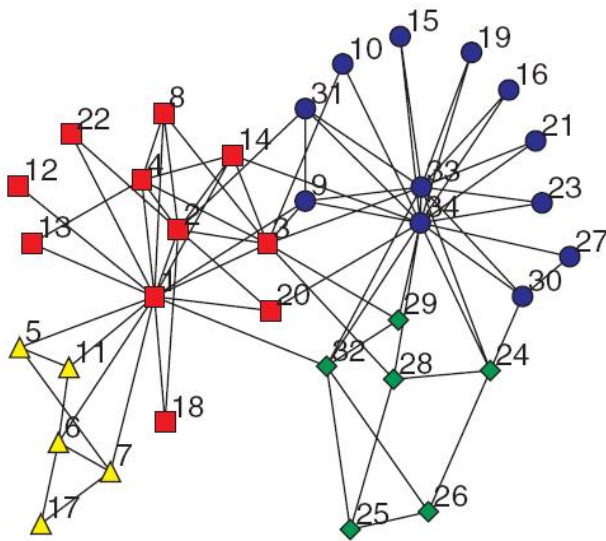
(b)

(a) $\lambda = 0.3$

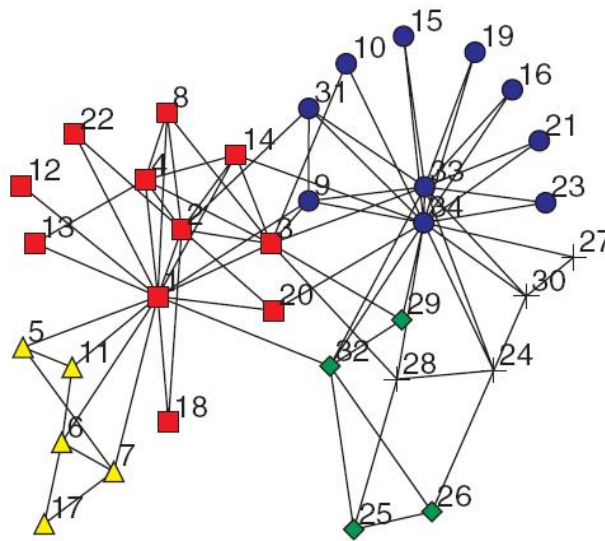
(b) $\lambda = 0.4$ and 0.5

(c) $\lambda = 0.6$ and 0.7

(d) $\lambda = 0.8$



(c)



(d)

展望

- ❧ 人工脑对生物脑已不只是功能模仿，而是机制模拟，人工脑与生物脑将具有相同的特性
- ❧ 多学科交叉、结合将开辟全新的智能计算领域，开辟很多新的研究方向
- ❧ 数学将大有用武之地，计算科学、统计学、模糊数学和系统学等将是智能计算理论的基石
- ❧ 智能计算将探索智能的新概念、新理论、新方法和新技术
- ❧ 21世纪的智能科学将有重大突破，取得重大成就